

Mémoire de fin d'études

Master 2 Intelligence Artificielle

Année 2025-2026

**Explicabilité et fiabilité des réseaux de neurones
pour la classification de tumeurs cérébrales**

Téodul Guiraud-Allègre

Tuteur : Yann Beaujeault-Taudière

Remerciements

Je remercie tout d'abord **l'École Hexagone** et son directeur **Xavier Auzanneau**, ainsi que **Pierre Uzarralde**, directeur du cursus Intelligence Artificielle, et **Dorine Raichon**, directrice du campus de Clermont-Ferrand, pour m'avoir permis de réaliser ce mémoire.

Je remercie aussi mon tuteur **Yann Beaujeault-Taudière** pour son aide apportée grâce à ses retours détaillés et ses corrections.

Je remercie également mes parents, **Joëlle Allègre** et **Éric Guiraud**, ma copine **Margaux Menou** et mon meilleur ami **Hamza Miftah** pour leur relecture attentive et leurs encouragements.

Je remercie enfin ma famille, en particulier mes grands-parents **Guy et Françoise Guiraud**, ainsi que mes proches, notamment **Léa Vergnol**, **Max Bernard** et **Thomas Clouvel**, pour leur soutien tout au long de mes études.

Résumé

L'imagerie des tumeurs cérébrales repose presque exclusivement sur l'IRM, et les modèles de deep learning atteignent aujourd'hui des performances comparables à celles d'un expert. Cependant, on ne sait pas vraiment sur quoi reposent leurs décisions ni quelle confiance leur accorder, ce qui pose problème en contexte clinique. Ce mémoire cherche à améliorer l'explicabilité et la fiabilité d'un modèle de classification d'IRM cérébrales à 4 classes (gliome, méningiome, tumeur pituitaire, absence de tumeur). J'ai d'abord entraîné un CNN de référence déterministe (ResNet18), puis construit une architecture bayésienne qui combine ce même ResNet18 avec une tête bayésienne (torchbnn, 50 passes Monte-Carlo). J'ai aussi testé une variante multimodale intégrant une estimation de l'âge cérébral, obtenue par un EfficientNet-B0 entraîné sur le dataset IXI. J'ai comparé ce BNN à une variante MC Dropout et validé les trois modèles sur le dataset externe Br35H.

J'ai constaté que 61.6 % des images de test étaient présentes dans le train. J'ai donc reconstruit un split propre (4 606 images d'entraînement, 1 149 de test, zéro doublon). Le passage au bayésien préserve les performances (AUC 0.995 contre 0.998 pour le CNN) tout en apportant une mesure d'incertitude par prédiction : les prédictions incorrectes sont 4.83 fois plus incertaines que les correctes. Ce BNN est mieux calibré que le CNN (ECE 0.0085 contre 0.0160). La variante multimodale dégrade la calibration (ECE 0.0276) et n'apporte pas de gain de performance.

En distribution, trier les prédictions par incertitude croissante permet de couvrir 87.8 % des erreurs en ne renvoyant que 20 % des images à un radiologue. torchbnn produit une incertitude en distribution 20 fois plus élevée que MC Dropout (σ médian 0.0217 contre 0.0010). MC Dropout est presque déterministe et ne permet pas de définir un seuil de triage clinique.

Ces chiffres correspondent à la seed de référence (42), qui s'est avérée la plus favorable. Sur les cinq seeds, le ratio $\sigma_{\text{incorrect}} / \sigma_{\text{correct}}$ reste supérieur à 2 (3.69 en moyenne) et le triage couvre toujours la majorité des erreurs (83.2 % en moyenne).

Au-delà de l'incertitude, les cartes Grad-CAM montrent sur quelles zones le modèle s'appuie, et leur variance sur 50 passes permet d'explorer où son attention varie le plus.

Ce travail montre qu'on peut rendre un modèle de classification IRM plus transparent et plus fiable sans sacrifier ses performances.

Mots-clés : classification IRM cérébrale, réseau bayésien, incertitude, Grad-CAM, calibration, explicabilité, MC Dropout, validation externe, qualité des données

Abstract

Brain tumor imaging relies almost exclusively on MRI, and deep learning models now reach performance comparable to that of an expert. Yet it usually remains unclear what their decisions rely on and how much confidence to place in them, which is a problem in a clinical setting. This thesis aims to improve the explainability and reliability of a four-class brain MRI classification model (glioma, meningioma, pituitary tumor, no tumor). I first trained a deterministic baseline CNN (ResNet18), then built a Bayesian architecture combining this same ResNet18 with a Bayesian head (torchbnn, 50 Monte-Carlo passes). I also tested a multimodal variant incorporating a brain-age estimate, produced by an EfficientNet-B0 trained on the IXL dataset. I compared this BNN to an MC Dropout variant and validated all three models on the external Br35H dataset.

I found that 61.6% of test images were present in the training set. I therefore rebuilt a clean split (4,606 training images, 1,149 test images, zero duplicates). Moving to a Bayesian model preserves performance (AUC 0.995 versus 0.998 for the CNN) while providing a per-prediction uncertainty measure: incorrect predictions are 4.83 times more uncertain than correct ones. This BNN is better calibrated than the CNN (ECE 0.0085 versus 0.0160). The multimodal variant degrades calibration (ECE 0.0276) and does not improve performance.

In-distribution, sorting predictions by uncertainty covers 87.8% of errors by sending only 20% of images to a radiologist. torchbnn produces an in-distribution uncertainty 20 times higher than MC Dropout (σ median 0.0217 versus 0.0010). MC Dropout is nearly deterministic and does not allow a clinical triage threshold to be defined.

These figures correspond to the reference seed (42), which turned out to be the most favorable. Across the five seeds, the $\sigma_{\text{incorrect}} / \sigma_{\text{correct}}$ ratio stays above 2 (3.69 on average) and triage still covers the majority of errors (83.2% on average).

Beyond uncertainty, Grad-CAM maps show which regions the model relies on, and their variance over 50 passes allows exploring where its attention varies the most.

This work shows that a brain MRI classification model can be made more transparent and more reliable without sacrificing performance.

Keywords: brain MRI classification, Bayesian neural network, uncertainty, Grad-CAM, calibration, explainability, MC Dropout, external validation, data quality

Table des matières

Table des matières	7
Liste des illustrations	11
Liste des tableaux	12
Liste des abréviations	13
I. Introduction	15
Phase exploratoire : prédiction du statut MGMT	17
II. Prérequis	19
II.A. Entraînement d'un réseau de neurones	19
II.B. Fondements mathématiques	20
II.B.1. Loi de Bayes	20
II.B.2. Réseau de neurones bayésien (BNN)	20
II.B.3. Inférence variationnelle	21
II.B.4. Divergence KL	21
II.B.5. Cross-entropie	21
II.B.6. Loss totale du BNN	22
II.B.7. MC Dropout	22
II.B.8. Incertitude aléatoire vs épistémique	23
II.C. Explicabilité visuelle	23
II.C.1. Grad-CAM	23
II.C.2. Grad-CAM sur un BNN	23
III. Présentation des datasets	25
III.A. Mendeley BT-MRI	25
III.A.1. Présentation	25
III.A.2. Contamination du dataset	26
III.A.3. Dataset propre	27
III.B. IXI	29
III.C. Br35H	30
III.C.1. Présentation	30
III.C.2. Nettoyage du dataset	31
IV. État de l'art	33
IV.A. Classification d'IRM cérébrales par deep learning	33
IV.A.1. Du traitement classique au transfer learning	33
IV.A.2. Le problème de la sur-confiance	33
IV.B. Réseaux bayésiens et incertitude	34
IV.B.1. MC Dropout	34
IV.B.2. Inférence variationnelle (torchbnn)	34

IV.C. Prédiction d'âge cérébral et fusion multimodale	34
IV.C.1. L'âge cérébral	34
IV.C.2. Fusion multimodale	34
IV.D. Contexte réglementaire	35
IV.E. Travaux existants sur les BNN en imagerie médicale	35
V. Méthodologie	37
V.A. Environnement et dépendances	37
V.B. Dataset : Mendeley BT-MRI	38
V.B.1. Prétraitement et augmentation	38
V.B.2. Gestion du déséquilibre de classes	38
V.C. Notebook 1 : CNN Baseline (ResNet18)	39
V.C.1. Architecture	39
V.C.2. Entraînement	41
V.D. Notebook 2 : Prédicteur d'âge (BrainAgeNet)	41
V.D.1. Extraction des coupes 2D depuis IXI	41
V.D.2. Architecture BrainAgeNet	42
V.D.3. Fonction de perte et optimiseur	43
V.D.4. Application au dataset tumoral	43
V.E. Notebook 3 : BNN	43
V.E.1. Architecture BNNMultimodalNet	43
V.E.2. La divergence KL	45
V.E.3. Optimiseur et scheduler	45
V.E.4. Inférence Monte-Carlo	46
V.E.5. BNN Image Only	46
V.F. Notebook 4 : Analyse de l'incertitude et calibration	47
V.F.1. Incertitude par classe	47
V.F.2. Calibration (ECE)	47
V.G. Notebook 5 : Grad-CAM et explicabilité visuelle	48
V.G.1. Implémentation de GradCAM	48
V.G.2. Grad-CAM avec 50 passes MC	49
V.H. Notebook 6 : Variante MC Dropout	49
V.I. Notebook 7 : Validation externe Br35H	50
V.I.1. Protocole	50
V.I.2. Détection du changement (shift) de distribution	51
V.I.3. Validation quantitative des Grad-CAM	51
V.J. Notebook 8 : Robustesse inter-seed	52
V.K. Notebook 9 : Analyses complémentaires	53
VI. Résultats	55
VI.A. CNN Baseline	55
VI.A.1. Choix du modèle ResNet	55
VI.A.2. Choix du learning rate	56

VI.B. Prédicteur d'âge (BrainAgeNet)	57
VI.C. Convergence du BNN	58
VI.D. Nombre de passes Monte-Carlo	59
VI.E. Matrice de confusion	60
VI.F. Incertitude par classe	61
VI.F.1. Distribution de l'incertitude	61
VI.F.2. Ratio $\sigma_{\text{incorrect}} / \sigma_{\text{correct}}$	62
VI.F.3. Confirmation des ratios par Bootstrap	63
VI.G. Calibration bayésienne	64
VI.H. Explicabilité visuelle : Grad-CAM	65
VI.I. Validation quantitative des Grad-CAM avec masques	67
VI.J. BNN Image Only vs multimodal	69
VI.K. Comparaison torchbnn vs MC Dropout	69
VI.L. Seuil de triage par incertitude	71
VI.M. Robustesse inter-seed	73
VI.N. Validation externe sur Br35H	74
VII. Discussion	77
VII.A. L'incertitude bayésienne est-elle utile ?	77
VII.B. torchbnn vs MC Dropout	77
VII.C. Pourquoi les modèles sont bien calibrés ?	77
VII.D. Apport de la feature clinique	78
VII.E. Comparaison avec les approches existantes	78
VII.F. Limites	79
VII.F.1. Limites méthodologiques	79
VII.F.2. Limites techniques	79
VII.F.3. Limites de généralisation	80
VII.F.4. Limites cliniques et éthiques	80
VII.G. Déploiement clinique	81
VII.H. Perspectives et travaux futurs	81
VIII. Conclusion	83
IX. Bibliographie	85
X. Annexes	89

Liste des illustrations

Figure 1 : Propagation avant (en bleu) et rétropropagation (en rouge) dans un réseau de neurones.	19
Figure 2 : À gauche : réseau classique avec des poids fixes. À droite : réseau bayésien avec des distributions sur les poids (courbes en orange).	21
Figure 3 : Illustration du MC Dropout. À gauche : réseau complet. À droite : réseau avec des neurones désactivés aléatoirement (en gris) pendant une passe.	22
Figure 4 : Exemples d'images IRM par classe du dataset Mendeley BT-MRI. De gauche à droite : gliome, méningiome, notumor, pituitaire.	26
Figure 5 : Exemples de doublons détectés entre le train et le test du dataset Mendeley BT-MRI. Chaque colonne montre une paire train/test. La distance pHash $d=0$ indique un doublon exact, $0 < d \leq 5$ un quasi-doublon visuellement proche.	26
Figure 6 : Distribution des distances pHash entre chaque image de test et son voisin le plus proche dans le train original.	27
Figure 7 : Distribution des classes dans le dataset Mendeley BT-MRI propre (train et test).	28
Figure 8 : Exemples de coupes 2D extraites depuis les volumes 3D du dataset IXI.	29
Figure 9 : Distribution des âges dans le dataset IXI (588 sujets sains).	29
Figure 10 : Exemples d'images du dataset Br35H. Ligne 1 : images avec tumeur (yes). Ligne 2 : images sans tumeur (no). Ligne 3 : exemple d'annotation de segmentation (image originale, masque, superposition).	30
Figure 11 : Distribution des classes dans le dataset Br35H.	31
Figure 12 : Distribution des distances pHash entre chaque image Br35H et son voisin le plus proche dans le train du dataset Mendeley BT-MRI propre. Les images à gauche du seuil ($d \leq 8$) ont été retirées de la validation OOD.	32
Figure 13 : Architecture DenseNet qui illustre le principe des skip connections. Tiré de (23).	40
Figure 14 : Fonction de perte avec et sans skip connections. À gauche : sans skip connections. À droite : avec skip connections. Tiré de (24).	40
Figure 15 : Comparaison des architectures ResNet18, ResNet34 et ResNet50. À gauche : AUC et balanced accuracy sur le jeu de test. À droite : performance en fonction du nombre de paramètres du modèle.	55
Figure 16 : Performances du CNN baseline selon le learning rate.	56
Figure 17 : Scatter plot âge prédit vs âge réel par BrainAgeNet sur le set de validation IXI. La couleur représente la densité locale des points (KDE, Kernel Density Estimation) : les zones claires concentrent le plus de prédictions.	57
Figure 18 : Courbe AUC pendant l'entraînement du BNN sur le set de validation.	58
Figure 19 : Courbes de loss, cross-entropie et divergence KL pendant l'entraînement du BNN.	58
Figure 20 : Convergence de l'incertitude épistémique en fonction du nombre de passes MC.	59
Figure 21 : Matrice de confusion du BNN.	60
Figure 22 : Stabilité de l'estimation de l'incertitude selon le nombre d'images par classe. À partir de 30 images, l'écart-type de l'estimation se stabilise pour toutes les classes.	61

Figure 23 : Distribution de l'incertitude épistémique du BNN par classe. Chaque boîte représente la distribution du σ sur 30 images aléatoires. La ligne centrale est la médiane, les bords de la boîte le premier et troisième quartile, les traits s'étendent jusqu'à 1.5 fois l'écart interquartile, et les points au-delà sont des valeurs extrêmes.	61
Figure 24 : Ratio $\sigma_{\text{incorrect}} / \sigma_{\text{correct}}$ par classe.	62
Figure 25 : Bootstrap du ratio $\sigma_{\text{incorrect}} / \sigma_{\text{correct}}$ par classe (1 000 itérations sur 25 % du jeu de test). La ligne pointillée à 1 représente l'absence de différence entre $\sigma_{\text{incorrect}}$ et σ_{correct}	63
Figure 26 : Reliability diagram du BNN.	64
Figure 27 : Comparaison Grad-CAM CNN vs BNN. Première ligne : images originales ; Deuxième ligne : cartes Grad-CAM du CNN ; Troisième ligne : cartes Grad-CAM du BNN. Les zones rouges correspondent aux activations les plus fortes, les zones bleues aux moins fortes.	65
Figure 28 : Carte de l'incertitude spatiale de l'attention du BNN. Première ligne : images originales ; Deuxième ligne : cartes Grad-CAM du BNN ; Troisième ligne : cartes σ . Pour les cartes σ , les zones claires indiquent une forte variabilité de l'attention entre les passes.	66
Figure 29 : Exemples qualitatifs de validation Grad-CAM sur Br35H. Pour chaque image : image originale, masque de segmentation, carte Grad-CAM et superposition.	68
Figure 30 : Comparaison des distributions σ in-distribution pour torchbnn et MC Dropout aux taux 0.2, 0.3 et 0.5.	70
Figure 31 : Courbe accuracy vs coverage sur le jeu de test Mendeley BT-MRI pour torchbnn. En gardant uniquement les prédictions les plus confiantes (faible σ), l'accuracy augmente progressivement. La ligne pointillée représente l'accuracy globale (0.964).	71
Figure 32 : Compromis charge de travail / sécurité pour torchbnn sur le jeu de test Mendeley BT-MRI. Chaque barre indique le pourcentage d'erreurs capturées en renvoyant les X % de prédictions les plus incertaines à un radiologue.	72
Figure 33 : Distribution de l'incertitude σ sur le Mendeley BT-MRI (in-distribution) et Br35H (OOD) pour le BNN torchbnn.	75

Liste des tableaux

Tableau 1 : Répartition des images par classe dans le dataset propre.	27
Tableau 2 : Dépendances et versions.	37
Tableau 3 : Validation quantitative des Grad-CAM sur 200 images annotées Br35H.	67
Tableau 4 : Récapitulatif des performances et temps d'inférence CNN Baseline / BNN Image Only / BNN Multimodal.	69
Tableau 5 : Comparaison torchbnn vs MC Dropout en in-distribution.	69
Tableau 6 : ECE et σ médian in-distribution du BNN Image Only (torchbnn) et de MC Dropout sur cinq seeds. La dernière ligne de chaque modèle donne la moyenne $\pm$ l'écart-type sur les cinq seeds.	73
Tableau 7 : Ratio $\sigma_{\text{incorrect}} / \sigma_{\text{correct}}$ et performances de triage du BNN Image Only (torchbnn) sur cinq seeds. La dernière ligne donne la moyenne $\pm$ l'écart-type sur les cinq seeds.	74
Tableau 8 : Performances OOD sur Br35H (binaire tumeur/pas tumeur).	74
Tableau 9 : Comparaison des taux de dropout sur Br35H. AUC val = AUC sur le Mendeley BT-MRI ; AUC OOD = AUC sur Br35H.	75

Liste des abréviations

AI Act : Artificial Intelligence Act, règlement européen sur l'intelligence artificielle (1).

AUC : Area Under the Curve, aire sous la courbe.

BalAcc : balanced accuracy, moyenne non pondérée de la précision par classe.

BNN : Bayesian Neural Network, réseau de neurones bayésien.

CNN : Convolutional Neural Network, réseau de neurones convolutif.

ECE : Expected Calibration Error, erreur de calibration attendue. Mesure l'écart moyen entre la confiance affichée par le modèle et sa précision réelle (0 = parfaitement calibré).

Grad-CAM : Gradient-weighted Class Activation Mapping, méthode de visualisation des zones d'activation d'un réseau de neurones.

IC95 : Intervalle de Confiance à 95 %.

in-dist : in-distribution, données de même distribution que celle utilisée à l'entraînement, par opposition aux données OOD (hors distribution).

IoU : Intersection over Union, chevauchement entre deux zones.

IRM : Imagerie par Résonance Magnétique.

KDE : Kernel Density Estimation, estimation de densité par noyau.

KL : Divergence de Kullback-Leibler.

MAE : Mean Absolute Error, erreur absolue moyenne.

MC : Monte-Carlo.

MLP : Multi-Layer Perceptron, perceptron multicouche.

MSE : Mean Squared Error, erreur quadratique moyenne.

OOD : Out-Of-Distribution, hors distribution.

OvR : One vs Rest, un contre tous.

pHash : Perceptual Hash, hachage perceptuel. Empreinte numérique calculée depuis le contenu visuel d'une image pour détecter les doublons.

R² : coefficient de détermination, mesure la qualité d'une régression (0 = aucune prédiction, 1 = prédiction parfaite).

SVM : Support Vector Machine, séparateur à vaste marge.

σ (sigma) : écart-type des prédictions MC, utilisé comme mesure d'incertitude.

I. Introduction

De tous les domaines professionnels auxquels j'ai pu m'intéresser, le secteur médical est celui qui m'a toujours le plus attiré. J'ai longtemps voulu pratiquer le métier de chirurgien mais les circonstances ne me l'ont pas permis, donc je me suis orienté vers l'informatique puis l'intelligence artificielle. Ce mémoire est une façon pour moi de faire le lien entre médecine et informatique, en espérant pouvoir y travailler dans le futur. J'ai donc choisi de travailler sur un sujet relatif à ce domaine et plus particulièrement sur les tumeurs cérébrales.

Les tumeurs cérébrales touchent plus de 320 000 personnes chaque année dans le monde (2) et on les détecte presque exclusivement à l'IRM (3). C'est une tâche difficile qui peut varier selon le radiologue qui l'effectue. Depuis quelques années, les algorithmes de deep learning ont montré qu'ils pouvaient atteindre des performances comparables à celles d'un expert sur la classification de ces images (4). Cependant, on ne sait pas comment ni pourquoi ils sont arrivés à ce résultat.

Par exemple, un CNN prédit « gliome » à 97 % de confiance, mais sur quelle zone de l'IRM s'est-il basé ? Est-ce qu'il est vraiment sûr, ou est-il surestimé ? Un médecin ne peut pas valider une décision qu'il ne comprend pas. Dans un contexte clinique, une « boîte noire » n'est pas acceptable.

La question de l'explicabilité des systèmes d'IA en médecine fait aujourd'hui partie des débats réglementaires et éthiques. Le règlement européen sur l'IA établit des exigences de transparence pour les systèmes à haut risque (1), dont les dispositifs médicaux. Il ne suffit plus qu'un modèle soit performant : **il doit être capable de justifier ses décisions de manière compréhensible pour les professionnels de santé.**

Comment améliorer l'explicabilité et la fiabilité d'un modèle de classification d'IRM cérébrales à 4 classes (gliome, méningiome, tumeur pituitaire et absence de tumeur), et dans quelle mesure l'incertitude bayésienne apporte-t-elle un avantage par rapport aux approches classiques ?

Dans ce mémoire, l'explicabilité désigne la capacité d'un modèle à justifier ses décisions de manière compréhensible : sur quoi s'appuie-t-il, quelle confiance lui accorder, et comment le mesurer ?

La contribution principale de ce travail porte sur l'explicabilité et la fiabilité. Des travaux existants utilisent déjà les BNN ou le Grad-CAM séparément en imagerie médicale, mais rarement l'association sur le même modèle. Ce travail combine cinq angles d'analyse sur le même modèle : une mesure d'incertitude par prédiction, une calibration évaluée via l'ECE, des cartes Grad-CAM calculées sur 50 passes Monte-Carlo, une comparaison expérimentale avec MC Dropout, et une validation externe sur Br35H incluant une évaluation quantitative des

Grad-CAM avec des masques de segmentation. J'explore aussi une carte de variance spatiale de l'attention, calculée comme l'écart-type pixel par pixel des 50 cartes Grad-CAM.

J'ai aussi découvert que 61.6 % des images de test étaient présentes dans le train. J'ai donc reconstruit un split propre et ré-entraîné tous les modèles sur ces données corrigées.

Les résultats montrent que trier les prédictions par incertitude permet de couvrir 87.8 % des erreurs en ne renvoyant que 20 % des images à un radiologue. MC Dropout ne permet pas ce triage car son incertitude est presque nulle en pratique.

Le mémoire commence par une phase exploratoire abandonnée, la prédiction du statut MGMT, qui m'a amené à recentrer le travail sur la classification de tumeurs. La suite est organisée de la façon suivante :

- **Chapitre II** : prérequis sur l'entraînement d'un réseau de neurones, les fondements mathématiques des BNN (loi de Bayes, inférence variationnelle, divergence KL) et le Grad-CAM
- **Chapitre III** : présentation des trois datasets utilisés (Mendeley BT-MRI, IXI, Br35H)
- **Chapitre IV** : état de l'art sur la classification d'IRM, les approches bayésiennes et le contexte réglementaire
- **Chapitre V** : méthodologie détaillée des 10 notebooks
- **Chapitre VI** : résultats expérimentaux
- **Chapitre VII** : discussion, limites et perspectives
- **Chapitre VIII** : conclusion

Phase exploratoire : prédiction du statut MGMT

Avant d'arriver au modèle présenté dans ce mémoire, j'ai exploré une première piste qui n'a pas abouti et que j'ai abandonnée pour passer à la classification de tumeurs : prédire le statut de méthylation du promoteur MGMT (O6-methylguanine-DNA methyltransferase) directement depuis les IRM. Je la documente ici car elle a fortement orienté le recentrage du mémoire sur la classification de tumeurs.

La méthylation MGMT

Le MGMT est un biomarqueur clé dans le glioblastome (forme la plus agressive de gliome) : quand le promoteur (la partie du gène qui régule son expression) est méthylé (chimiquement modifié de façon à réduire l'expression du gène), les patients répondent mieux à la chimiothérapie, avec une survie médiane de 21.7 mois quand elle est associée à la radiothérapie, contre 15.3 mois avec la radiothérapie seule (5). Actuellement ce statut est déterminé par biopsie, ce qui est invasif et implique un processus qui dure plusieurs jours.

Le dataset UCSF-PDGM

Le dataset UCSF-PDGM contient 501 patients avec des IRM cérébrales et leur statut MGMT annoté (6). C'est un dataset de 142 Go avec des volumes IRM 3D, ce qui le rend beaucoup plus lourd à traiter que les datasets 2D.

Tests effectués

J'ai entraîné un CNN 3D sur ce dataset, avec et sans masque de segmentation tumoral (une annotation qui délimite précisément la zone tumorale sur l'IRM).

Résultats

Dans tous les cas l'AUC est restée autour de 0.45-0.50, proche du hasard. Le temps d'entraînement était considérable et les résultats insuffisants. J'ai décidé de passer à la classification de tumeurs, plus directement liée à l'explicabilité qui était au cœur de mon sujet. L'état de l'art qui suit se concentre donc uniquement sur ce second axe de travail.

II. Prérequis

II.A. Entraînement d'un réseau de neurones

L'entraînement se déroule en **epochs** (passages complets sur toutes les données). À chaque epoch, les images sont traitées par groupes appelés **batches** (figure 1).

Pour chaque batch :

- **La propagation avant (forward propagation)** fait passer les images à travers les couches du réseau pour produire une prédiction
- **La fonction de perte (appelée loss)** mesure l'erreur entre cette prédiction et la réalité
- Cette erreur est dérivée pour obtenir le **gradient** de chaque poids
- **La rétropropagation (backward propagation)** remonte ce gradient couche par couche
- **L'optimiseur** utilise ce gradient pour ajuster les poids

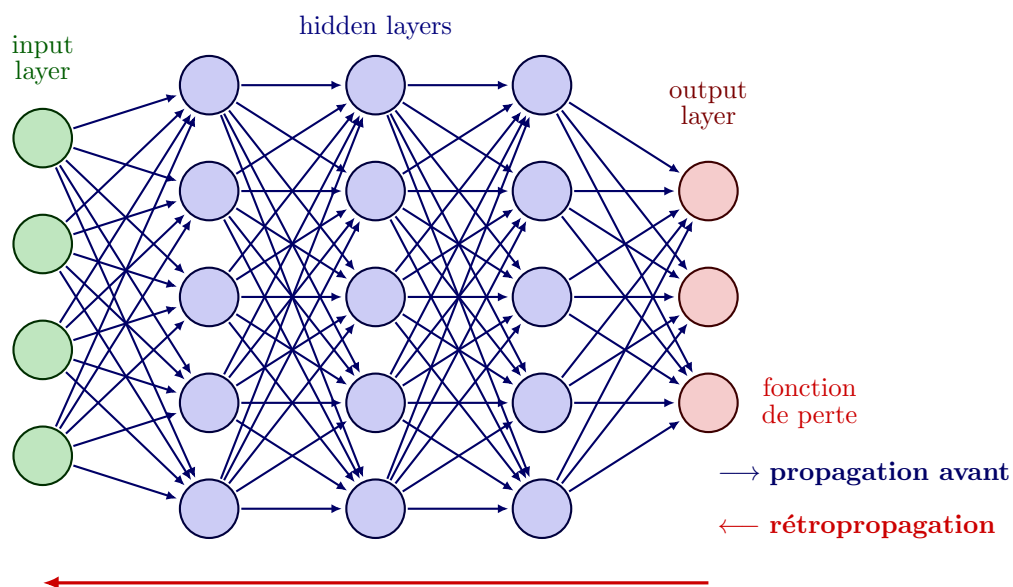


Figure 1 : Propagation avant (en bleu) et rétropropagation (en rouge) dans un réseau de neurones.

Le learning rate (taux d'apprentissage) contrôle à quel point les poids sont ajustés à chaque batch. **Un scheduler (planificateur)** le fait varier pendant l'entraînement, la plupart du temps en le réduisant progressivement.

II.B. Fondements mathématiques

II.B.1. Loi de Bayes

La loi de Bayes permet de mettre à jour une probabilité en fonction de nouvelles observations. On part d'une probabilité initiale appelée **prior**, et on obtient une probabilité mise à jour appelée **posterior** :

$$P(A | B) = P(B | A) \times P(A) / P(B)$$

On lit $P(A|B)$: probabilité de A sachant B.

Avec :

- **P(A)** : le **prior**, la probabilité de A avant d'avoir observé B
- **P(B | A)** : la probabilité d'observer B si A est vrai
- **P(A | B)** : le **posterior**, la probabilité de A après avoir observé B
- **P(B)** : la probabilité d'observer B quelle que soit la cause

Supposons l'événement A = avoir la grippe et B = observer une perte d'odorat. On obtient :

$$P(\text{grippe} | \text{perte d'odorat}) = P(\text{perte d'odorat} | \text{grippe}) \times P(\text{grippe}) / P(\text{perte d'odorat})$$

II.B.2. Réseau de neurones bayésien (BNN)

Dans un réseau classique, chaque poids est un nombre fixe. Dans un BNN, on applique ce même principe de Bayes aux poids du réseau : chaque poids devient une **variable aléatoire** qui suit une distribution (figure 2), c'est cette distribution qu'on cherche à apprendre. En notant w les poids et D les données, ce qu'on cherche c'est **$P(w | D)$** : la meilleure distribution des poids en considérant les données qu'on a observées. En appliquant la loi de Bayes :

$$P(w | D) = P(D | w) \times P(w) / P(D)$$

- **P(w)** : c'est le **prior**, la supposition initiale sur les poids avant de voir les données. Ce n'est pas une probabilité au sens entre 0 et 1, c'est une densité de probabilité qui décrit quelles valeurs sont plus ou moins plausibles. On choisit $N(0, 0.1)$, la valeur par défaut de torchbnn : on suppose que les poids sont proches de zéro, avec un écart-type de 0.1.
- **P(D | w)** : la probabilité d'observer ces données si les poids avaient ces valeurs. Si les poids sont bons, cette probabilité est haute. Le lien avec la cross-entropie est présenté en sous-section II.B.5.
- **P(D)** : la probabilité d'observer ces données quelle que soit la cause. Ce terme est impossible à calculer exactement car il nécessiterait de considérer toutes les combinaisons de poids possibles.

- $P(\mathbf{w} \mid \mathbf{D})$: c'est le **posterior**, ce qu'on cherche. La distribution des poids mise à jour après avoir vu les données.

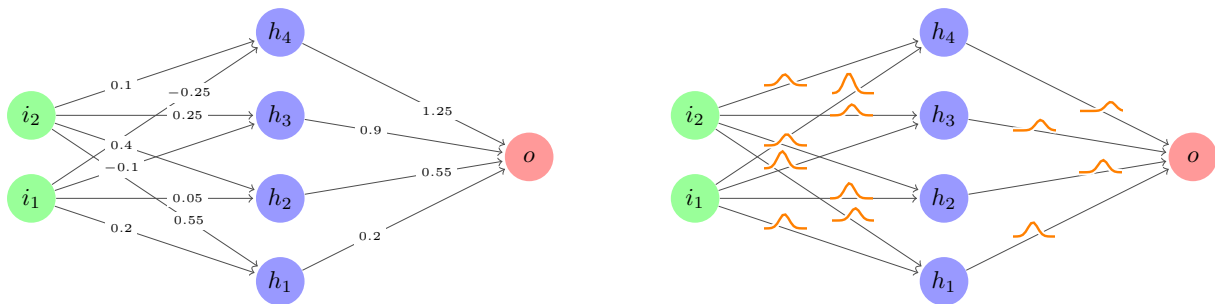


Figure 2 : À gauche : réseau classique avec des poids fixes. À droite : réseau bayésien avec des distributions sur les poids (courbes en orange).

À l'inférence, on tire un ensemble de poids depuis ces distributions à chaque passe, ce qui produit une prédiction légèrement différente à chaque fois. **La variance entre ces prédictions mesure l'incertitude, qu'on exploite dans ce mémoire pour identifier les prédictions peu fiables et améliorer la transparence du modèle.**

II.B.3. Inférence variationnelle

Calculer $P(\text{poids} \mid \text{données})$ exactement est impossible. L'inférence variationnelle remplace cette vraie distribution par une distribution gaussienne simple (en cloche, caractérisée par une moyenne et un écart-type) dont on optimise les paramètres. Sans contrainte supplémentaire, les poids appris s'éloigneraient trop du prior. C'est là qu'intervient la divergence KL.

II.B.4. Divergence KL

Sans contrainte, l'optimiseur ferait tendre l'écart-type des poids vers zéro pour stabiliser la loss, et le modèle redeviendrait déterministe. La **divergence KL (Kullback-Leibler)** mesure l'écart entre la distribution des poids apprise et le prior. Plus les deux sont proches, plus la KL est faible. Elle est ajoutée comme pénalité dans la loss : si l'écart-type appris s'approche de zéro, il s'éloigne du prior $N(0, 0.1)$ et la pénalité augmente.

II.B.5. Cross-entropie

La cross-entropie est l'opposé du logarithme de la vraisemblance, donc minimiser la cross-entropie revient à maximiser la vraisemblance $P(\mathbf{D} \mid \mathbf{w})$. Plus la prédiction est éloignée de la vraie classe, plus la pénalité est élevée.

II.B.6. Loss totale du BNN

Dans un BNN, la loss totale combine les deux :

$$\text{loss} = \text{cross-entropie} + 0.01 \times \text{KL}$$

La cross-entropie pousse le modèle à bien classifier, la KL empêche les poids de trop s'éloigner du prior. Le coefficient 0.01 contrôle l'équilibre entre les deux. C'est une valeur empirique couramment utilisée : trop élevé, il écraserait la cross-entropie et le modèle apprendrait mal à classifier.

II.B.7. MC Dropout

Le dropout est une technique qui désactive aléatoirement certains neurones pendant l'entraînement, habituellement utilisée pour éviter le sur-apprentissage (quand le modèle mémorise les images d'entraînement plutôt que d'apprendre à généraliser). Gal & Ghahramani (7) ont montré qu'en laissant le dropout actif pendant les prédictions (et pas seulement pendant l'entraînement), chaque prédiction devient légèrement différente car des neurones différents sont désactivés à chaque fois. En répétant plusieurs prédictions sur la même image on peut estimer une incertitude. C'est le principe de la méthode Monte-Carlo, d'où le nom MC (Monte-Carlo) Dropout.

Le dropout désactive aléatoirement certains neurones à chaque passe (figure 3).

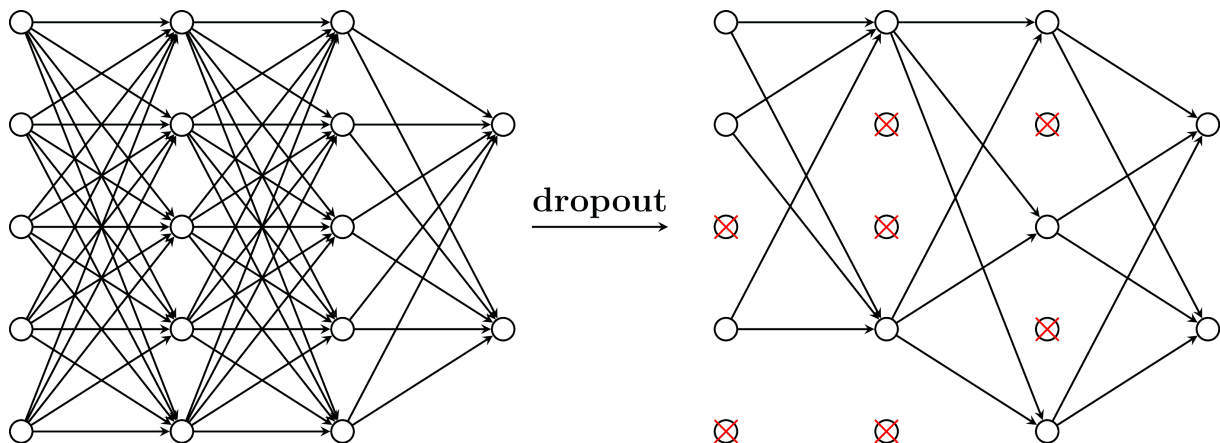


Figure 3 : Illustration du MC Dropout. À gauche : réseau complet. À droite : réseau avec des neurones désactivés aléatoirement (en gris) pendant une passe.

Contrairement à l'inférence variationnelle, il n'y a pas de prior ni de divergence KL : c'est uniquement le masque aléatoire du dropout qui introduit la variabilité entre les prédictions.

II.B.8. Incertitude aléatoire vs épistémique

Il y a deux types d'incertitude en deep learning (8).

L'incertitude aléatoire vient des données elles-mêmes : bruit dans l'image, variations naturelles entre patients, etc. Elle ne diminue pas avec plus de données. La quantifier est plus difficile : il faudrait modéliser explicitement le bruit dans les données, par exemple en ajoutant un bruit gaussien sur les images. Kendall & Gal (8) proposent une telle approche, mais je ne l'ai pas implémentée dans ce mémoire.

L'incertitude épistémique vient du modèle : elle diminue quand on a plus de données ou un meilleur modèle. Dans mon cas, l'écart-type des probabilités softmax sur plusieurs passages mesure l'incertitude épistémique : comme seule la tête est bayésienne, la variabilité entre passes vient uniquement du tirage aléatoire des poids de cette tête, pas du bruit des données.

II.C. Explicabilité visuelle

II.C.1. Grad-CAM

Grad-CAM (Gradient-weighted Class Activation Mapping) (9) génère une carte de chaleur superposée à l'image, qui montre sur quelles zones le modèle s'est basé pour prédire une classe.

En contexte médical, cela permet de vérifier que le modèle cible bien la tumeur et non une zone sans rapport avec celle-ci.

II.C.2. Grad-CAM sur un BNN

Quand on applique Grad-CAM à un BNN avec 50 passes MC, on obtient 50 cartes différentes puisque les poids changent à chaque passe. La moyenne de ces cartes donne une carte d'attention, et l'écart-type pixel par pixel donne une carte d'incertitude : les zones avec un fort écart-type sont celles où l'attention varie le plus entre les passes.

III. Présentation des datasets

Les images sont découpées en trois ensembles :

- **Train** : les images utilisées pour l'entraînement
- **Val (validation)** : des images retirées du train pour détecter le sur-apprentissage
- **Test** : les images utilisées pour l'évaluation finale

III.A. Mendeley BT-MRI

III.A.1. Présentation

Le dataset Brain Tumor MRI (10) contient 12 064 images IRM en niveaux de gris, réparties en 3 types de tumeurs et une classe d'absence de tumeur. Il est disponible sur Mendeley Data. La provenance exacte des images n'est pas documentée : le dataset ne contient ni identifiants patients ni métadonnées cliniques. Je l'appelle Mendeley BT-MRI par la suite.

Les 4 classes :

- **Gliome** : tumeur qui se développe dans le tissu cérébral
- **Méningiome** : tumeur qui se développe sur les membranes entourant le cerveau
- **Tumeur pituitaire** ou hypophysaire (que j'appellerai parfois pituitaire pour alléger, d'après le nom de la classe « pituitary » dans le dataset) : tumeur qui se développe dans l'hypophyse
- **Notumor** (absence de tumeur)

Chaque type a des traitements très différents, une erreur de classification aurait donc des conséquences directes sur le patient. Dans les figures et tableaux, ces classes apparaissent sous leur nom anglais : glioma, meningioma, notumor, pituitary.

La figure 4 montre, pour chaque classe, une image du train et une du test :

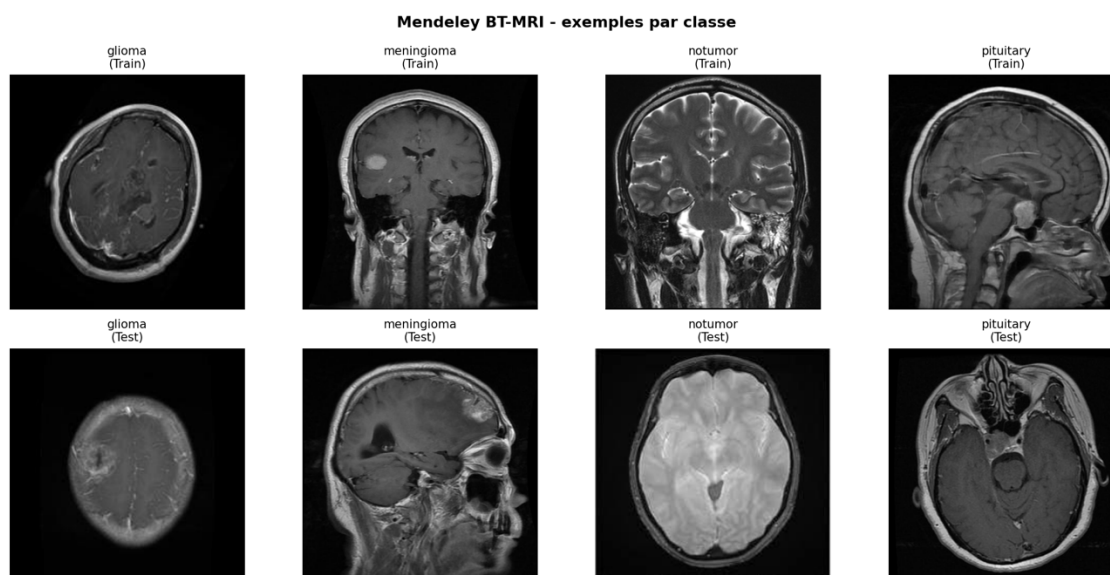


Figure 4 : Exemples d'images IRM par classe du dataset Mendeley BT-MRI. De gauche à droite : gliome, méningiome, notumor, pituitaire.

III.A.2. Contamination du dataset

J'ai découvert que 1 486 des 2 414 images du test original, soit 61.6 %, étaient des doublons exacts d'images du train. Je les ai détectées par hachage perceptuel (pHash) (11) : cette méthode génère une empreinte numérique depuis le contenu visuel d'une image, et plus deux images se ressemblent, plus leurs empreintes sont proches. La figure 5 montre des exemples de ces doublons et quasi-doublons :

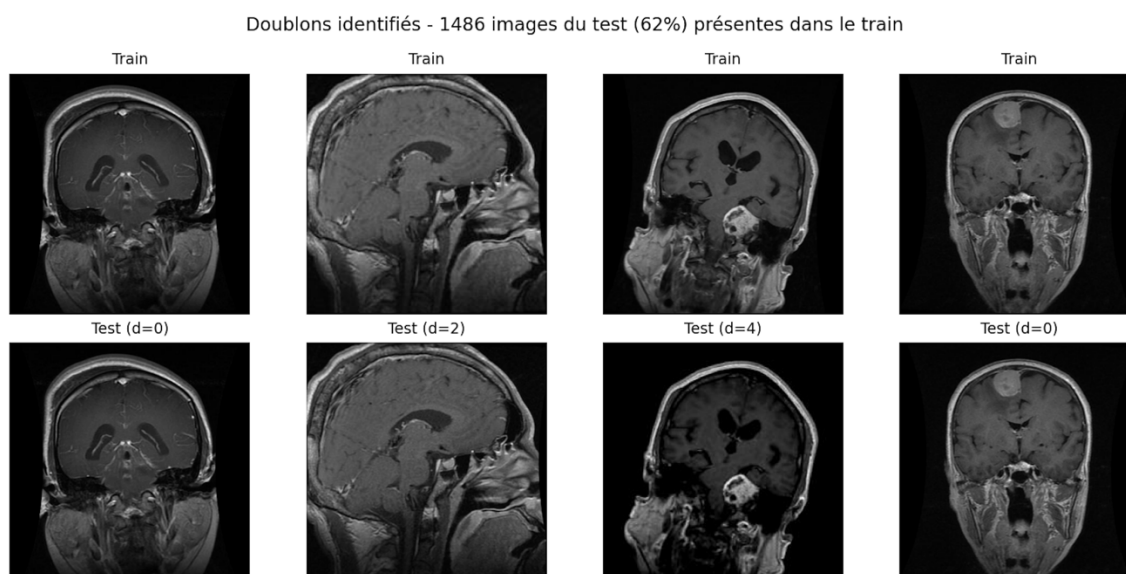


Figure 5 : Exemples de doublons détectés entre le train et le test du dataset Mendeley BT-MRI. Chaque colonne montre une paire train/test. La distance pHash $d=0$ indique un doublon exact, $0 < d \leq 5$ un quasi-doublon visuellement proche.

La figure 6 montre, pour chaque image de test, la distance pHash à l'image du train la plus proche (le pic à zéro correspond aux doublons exacts) :

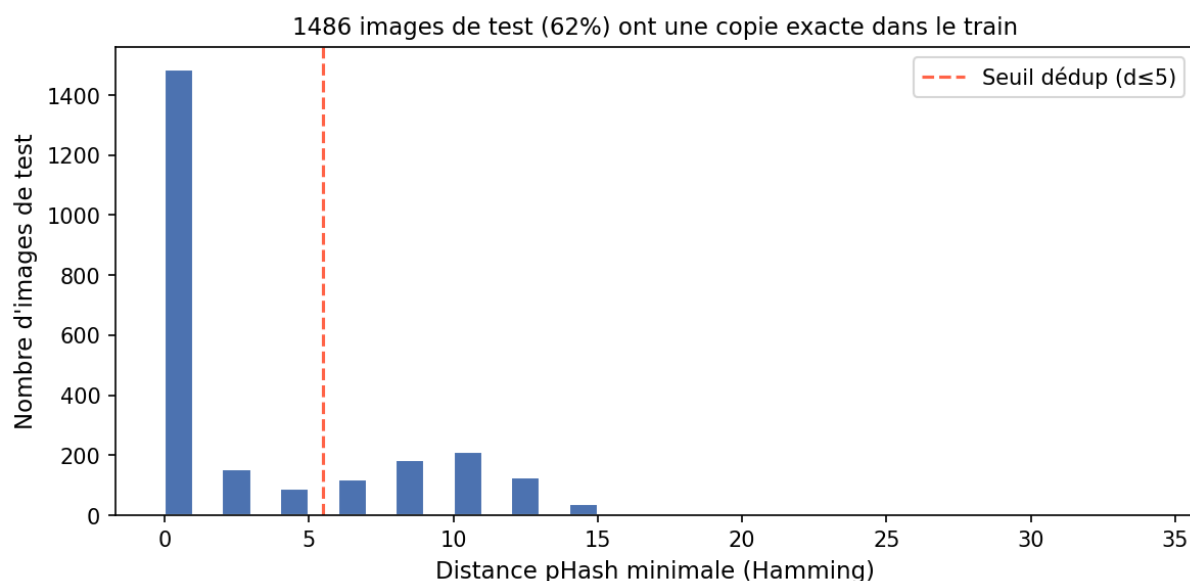


Figure 6 : Distribution des distances pHash entre chaque image de test et son voisin le plus proche dans le train original.

III.A.3. Dataset propre

J'ai remis en commun toutes les images, puis dédoublé par hachage perceptuel au seuil $d \leq 5$. La déduplication retire 6 309 images sur 12 064, soit plus de la moitié du dataset. Je refais ensuite un split 80/20 en gardant les images proches détectées par pHash du même côté, soit en train soit en test, pour qu'aucune ne se retrouve dans les deux. Le test d'acceptation confirme zéro doublon.

Le dataset propre compte 4 606 images d'entraînement et 1 149 de test.

Split	Total	Gliome	Méningiome	Notumor	Pituitaire
Train	4 606	1 687	1 148	592	1 179
Test	1 149	421	286	147	295

Tableau 1 : Répartition des images par classe dans le dataset propre.

Je réserve 15 % par classe (689 images) comme set de validation. Le test ne sert qu'à l'évaluation finale.

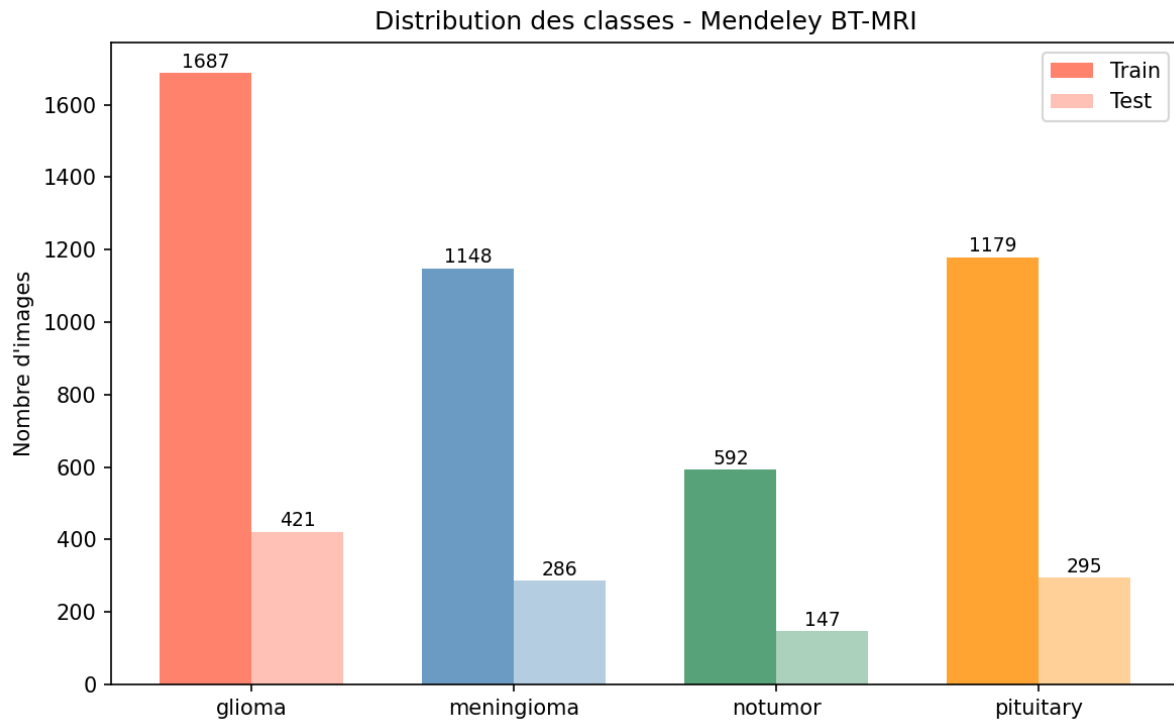


Figure 7 : Distribution des classes dans le dataset Mendeley BT-MRI propre (train et test).

Les classes ne sont pas équilibrées (figure 7) : notumor reste la plus petite avec 592 images en train contre 1 687 pour le gliome. Ce déséquilibre est géré par un `WeightedRandomSampler` à l'entraînement (V.B.2).

La déduplication supprime les doublons visuels mais ne garantit pas l'absence de fuite au niveau des patients. Deux images d'un même patient peuvent être visuellement différentes, donc non détectées par pHash et réparties entre le train et le test.

III.B. IXI

Le dataset IXI contient 588 sujets sains, chacun avec une IRM T1 (un type d'acquisition IRM mettant en valeur l'anatomie cérébrale) ainsi que leur âge réel compris entre 20 et 86 ans. C'est sur ce dataset que j'ai entraîné mon modèle de prédiction de l'âge.

Voici des exemples de coupes 2D extraites des IRM :

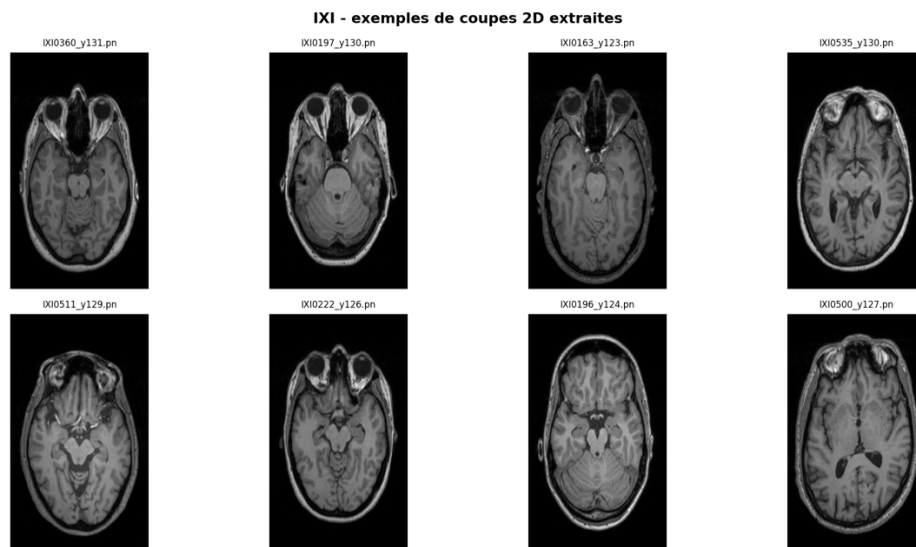


Figure 8 : Exemples de coupes 2D extraites depuis les volumes 3D du dataset IXI.

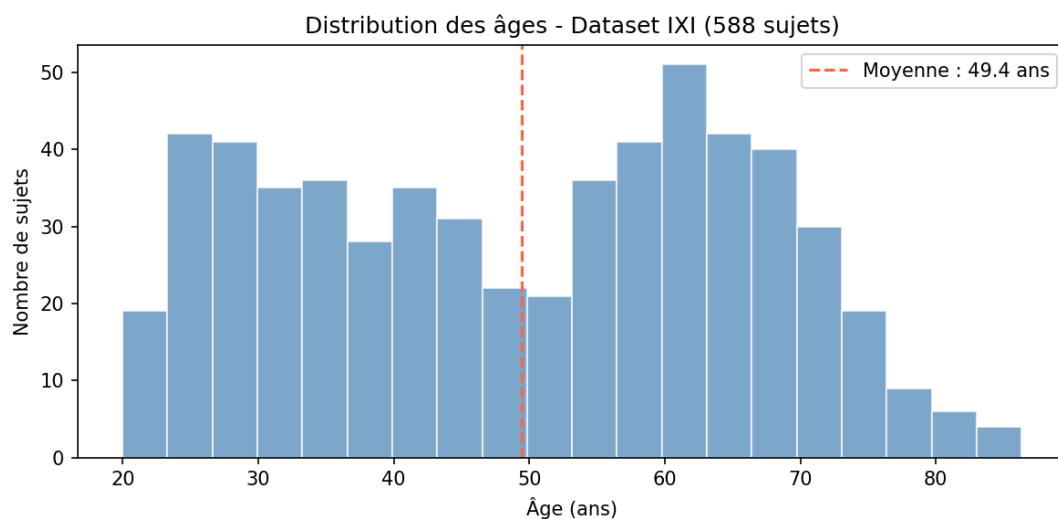


Figure 9 : Distribution des âges dans le dataset IXI (588 sujets sains).

La distribution est relativement uniforme entre 20 et 86 ans, sans concentration excessive sur les âges médians (figure 9).

III.C. Br35H

III.C.1. Présentation

Le dataset Br35H (12) contient 3 000 images IRM en deux classes : 1 500 images avec tumeur et 1 500 sans tumeur. Il est disponible sur Kaggle. Contrairement au dataset Mendeley BT-MRI, il ne distingue pas le type de tumeur. Ce dataset est utilisé pour deux choses dans ce mémoire : tester la robustesse des modèles sur des données jamais vues pendant l'entraînement, et valider quantitativement les cartes Grad-CAM grâce aux annotations de segmentation fournies avec le dataset.

Voici des exemples d'images du dataset, avec et sans tumeur :

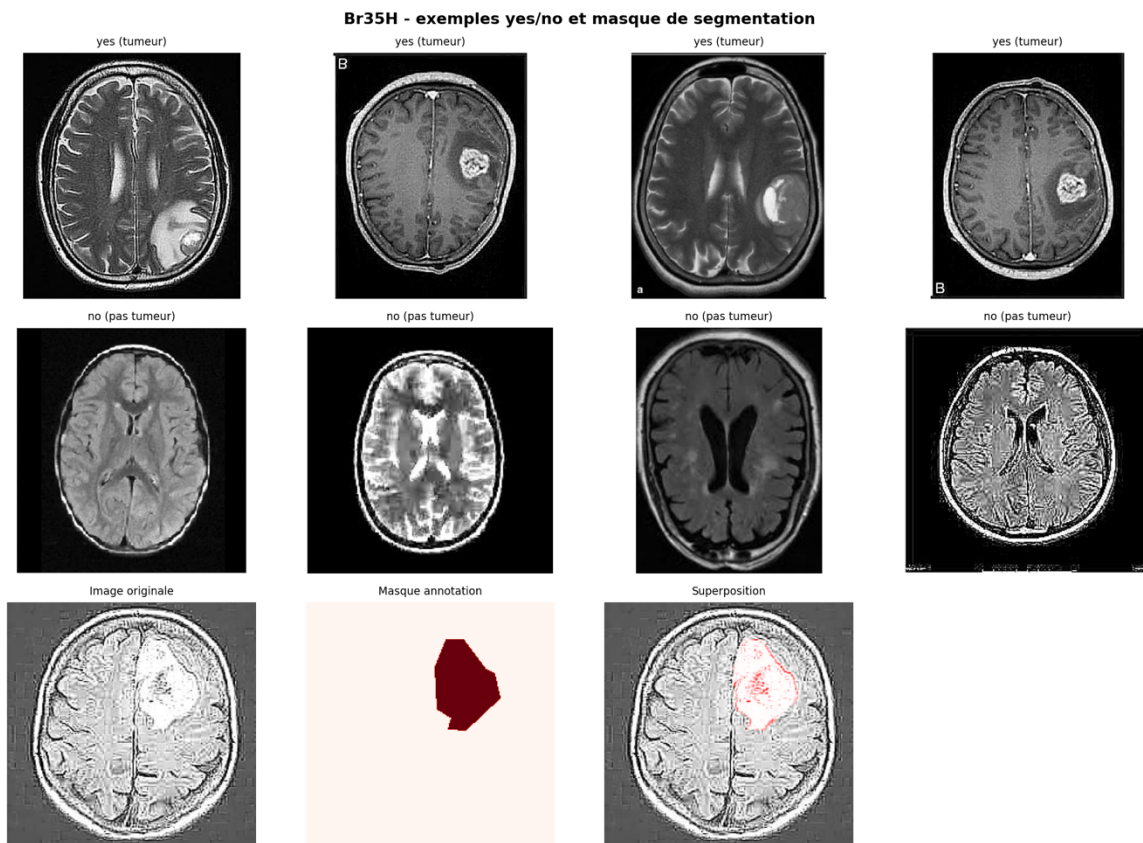


Figure 10 : Exemples d'images du dataset Br35H. Ligne 1 : images avec tumeur (yes). Ligne 2 : images sans tumeur (no). Ligne 3 : exemple d'annotation de segmentation (image originale, masque, superposition).

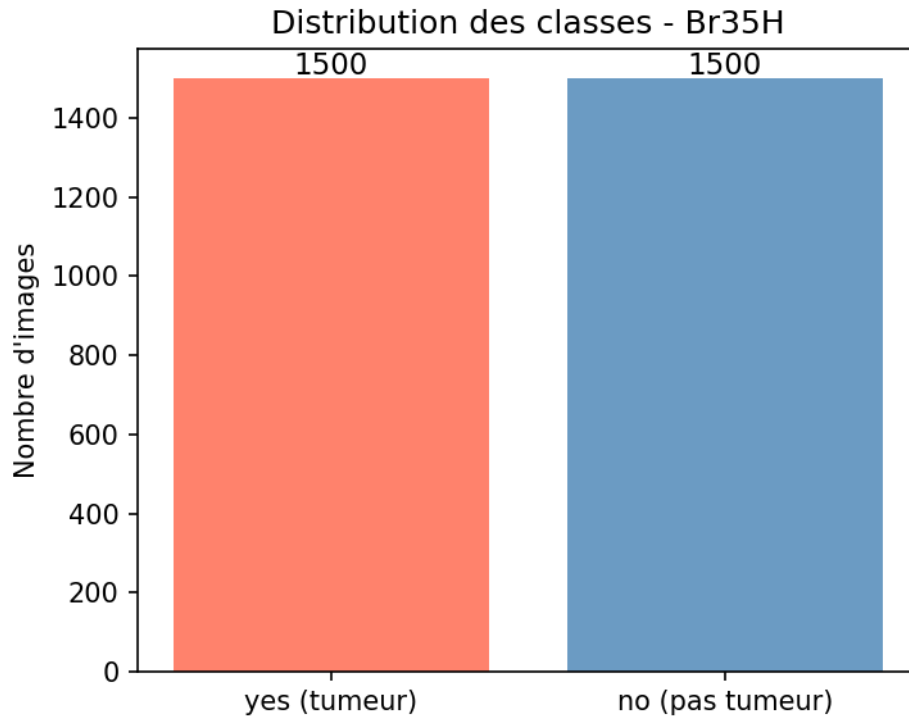


Figure 11 : Distribution des classes dans le dataset Br35H.

Les deux classes sont parfaitement équilibrées (figure 11), contrairement au dataset Mendeley BT-MRI (figure 7).

III.C.2. Nettoyage du dataset

En comparant les images Br35H avec le train propre du Mendeley BT-MRI par hachage perceptuel, j'ai retiré les images déjà vues pendant l'entraînement. Pour la validation hors distribution (OOD, Out-Of-Distribution), j'ai choisi un seuil plus strict ($d \leq 8$, on retire plus d'images proches) qu'à la déduplication ($d \leq 5$), afin de m'assurer que le jeu OOD ne contient aucune image visuellement proche du train. Sans ce nettoyage, le modèle aurait été évalué sur des images vues à l'entraînement, ce qui aurait faussé les résultats de généralisation.

- Doublons exacts ($d=0$) : 666 images (22.2 %)
- Quasi-doublons ($d \leq 8$) : 1 407 images (46.9 %), dont 188 yes (12.5 %) et 1 219 no (81.3 %)
- Conservés pour la validation OOD : 1 593 images

Le déséquilibre entre les classes est marqué : 81.3 % des images sans tumeur ont été retirées contre seulement 12.5 % des images avec tumeur (figure 12).

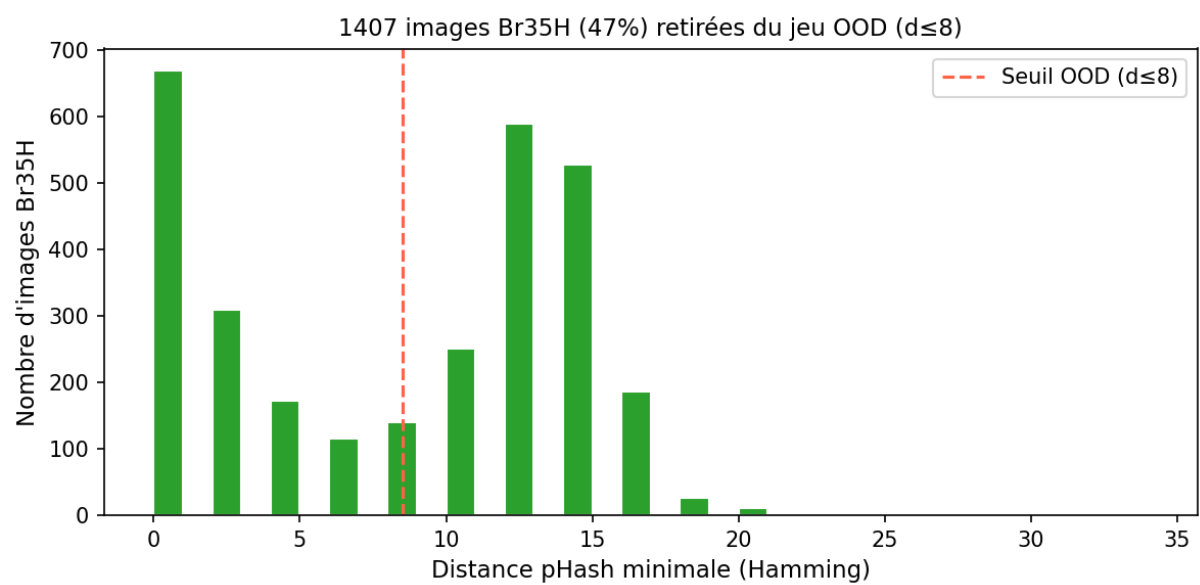


Figure 12 : Distribution des distances *pHash* entre chaque image *Br35H* et son voisin le plus proche dans le train du dataset *Mendeley BT-MRI* propre. Les images à gauche du seuil ($d \leq 8$) ont été retirées de la validation OOD.

IV. État de l'art

Cette partie couvre les travaux qui ont orienté les choix de ce mémoire : classification d'IRM par deep learning, réseaux bayésiens et incertitude, fusion multimodale en imagerie médicale, et contexte réglementaire.

IV.A. Classification d'IRM cérébrales par deep learning

IV.A.1. Du traitement classique au transfer learning

Avant le deep learning, classifier des IRM voulait dire extraire des features (caractéristiques) à la main puis les passer dans des algorithmes de classification comme des SVM (Support Vector Machines) ou des random forests (forêts d'arbres de décision). Le réseau AlexNet (13), premier CNN à avoir prouvé l'efficacité du deep learning à grande échelle en 2012, a montré que l'on pouvait apprendre ces features directement depuis les pixels.

Fine-tuner un CNN pré-entraîné sur ImageNet surpasse l'entraînement depuis zéro même en imagerie médicale (14). C'est le principe du transfer learning : plutôt que d'apprendre tout depuis zéro, on part de poids déjà appris sur une grande base d'images.

Sur la classification de tumeurs cérébrales, les CNN fine-tunés atteignent actuellement des précisions entre 94 % et 99 % sur des datasets publics à 3 ou 4 classes (4). Pour cette raison, j'ai décidé d'utiliser un ResNet (15), pré-entraîné sur ImageNet.

IV.A.2. Le problème de la sur-confiance

Actuellement, on sait faire des prédictions correctes, cependant on ne peut pas toujours se fier à la confiance affichée : les CNN modernes sont souvent sur-confiants. Leur probabilité softmax, la fonction qui transforme la sortie du réseau en distribution de probabilités sur les classes, n'est pas toujours une mesure fiable.

Ce problème a été mesuré par Guo et al. (16) avec l'ECE (Expected Calibration Error, qui mesure l'écart moyen entre la confiance affichée et la précision réelle) sur des architectures comme ResNet ou DenseNet : les ECE sont entre 0.04 et 0.08 sur des datasets standards. En contexte clinique, un modèle affichant 98 % de confiance tout en étant incorrect peut induire le médecin en erreur, j'ai donc décidé d'explorer dans quelle mesure un BNN peut améliorer cette limite.

IV.B. Réseaux bayésiens et incertitude

IV.B.1. MC Dropout

Le MC Dropout, présenté en prérequis, est plus simple à mettre en place, mais c'est une approximation indirecte : le dropout a été conçu comme technique de régularisation, pas comme outil de modélisation de l'incertitude. L'incertitude estimée dépend fortement du taux de dropout choisi, un hyperparamètre arbitraire. J'ai donc préféré utiliser l'inférence variationnelle. Je confirme expérimentalement ce choix dans le notebook 6, présenté dans le chapitre VI.

IV.B.2. Inférence variationnelle (torchbnn)

Le prior $N(0, 0.1)$ et le poids KL (0.01) sont les valeurs par défaut de torchbnn, qui implémente l'inférence variationnelle de Blundell et al. (17) via ses couches BayesLinear, que j'utilise pour la tête bayésienne de mon modèle.

IV.C. Prédiction d'âge cérébral et fusion multimodale

IV.C.1. L'âge cérébral

Le dataset Mendeley BT-MRI ne contient aucune information sur les patients (âge, sexe, antécédents médicaux et familiaux, données cliniques et biologiques). Ajouter une variable clinique à un modèle d'image peut réduire sa dépendance aux biais visuels du dataset. J'ai choisi d'estimer l'âge à partir de l'image elle-même, afin d'obtenir une feature clinique à fournir au BNN. Cole et al. (18) ont montré que l'âge cérébral estimé depuis une IRM est un biomarqueur fiable.

IV.C.2. Fusion multimodale

J'utilise une fusion intermédiaire : plutôt que de traiter l'image et l'âge séparément jusqu'à la prédiction finale, je les combine avant la tête bayésienne. L'âge prédit est transformé en 32 features par un MLP (Multi-Layer Perceptron, réseau de neurones classique à couches successives), concaténées avec les 512 features visuelles du CNN. La tête bayésienne utilise ce vecteur de 544 features pour produire une prédiction. Cette architecture sert de comparaison avec le BNN Image Only, présenté dans le chapitre VI.

IV.D. Contexte réglementaire

L'AI Act (1) classe les systèmes d'aide au diagnostic médical comme systèmes à haut risque et impose des exigences de transparence et de supervision humaine. Ce qui veut dire qu'un modèle qui prédit sans expliquer n'est plus acceptable. La combinaison incertitude bayésienne et Grad-CAM répond à ces exigences : l'incertitude mesure le niveau de confiance de la prédiction, les cartes d'attention montrent sur quelles zones de l'image la prédiction s'appuie. L'ajout de variables cliniques comme l'âge peut aussi aider à réduire la dépendance aux biais visuels du dataset.

IV.E. Travaux existants sur les BNN en imagerie médicale

Le MC Dropout est l'approche la plus utilisée pour quantifier l'incertitude en imagerie médicale. Roy et al. (19) l'ont appliqué à la segmentation d'IRM cérébrale et ont montré que les cartes d'incertitude identifient bien les zones difficiles à segmenter.

Nair et al. (20) ont appliqué la même approche à la détection de lésions de sclérose en plaques et ont montré que filtrer les prédictions les plus incertaines améliore les performances.

Plus proche de mon travail, Natekar et al. (21) ont utilisé MC Dropout sur un réseau de segmentation de tumeurs cérébrales et ont montré que les cartes d'incertitude correspondent aux zones difficiles pour les experts. La différence par rapport à mon travail est qu'ils travaillent sur de la segmentation avec MC Dropout, alors que j'utilise torchbnn sur de la classification, avec une comparaison expérimentale entre les deux méthodes et une validation externe sur Br35H.

V. Méthodologie

Le travail est organisé en 10 notebooks Python 3.9. L'environnement d'exécution est un MacBook Pro avec puce Apple M5 (10 cœurs CPU, 24 Go de RAM).

Le notebook 0 déduplique le dataset et construit le split propre décrit dans le chapitre III. Le notebook 1 entraîne le CNN baseline (modèle de référence). Le notebook 2 entraîne le prédicteur d'âge et génère les features cliniques. Le notebook 3 entraîne le BNN à partir des poids CNN et des features cliniques. Les notebooks 4 et 5 analysent les résultats. Le notebook 6 entraîne la variante MC Dropout pour comparer avec torchbnn. Le notebook 7 évalue les modèles sur le dataset externe Br35H et valide les Grad-CAM avec les masques de segmentation. Le notebook 8 réentraîne le BNN Image Only et MC Dropout sur cinq seeds. Le notebook 9 regroupe les analyses complémentaires.

Les extraits de code dans cette partie sont simplifiés pour la lisibilité. Les notebooks complets sont référencés en annexe et accessibles sur le dépôt GitHub : <https://github.com/teodulga/memoire-m2-bnn-irm>

V.A. Environnement et dépendances

Les librairies utilisées et leurs versions exactes :

Librairie	Version
Python	3.9
torch	2.8.0
torchvision	0.23.0
torchbnn	1.2
nibabel	5.3.3
scikit-learn	1.6.1
numpy	2.0.2
matplotlib	3.9.4
Pillow	11.3.0
pandas	2.3.3
opencv	4.13.0
scipy	1.13.1

Tableau 2 : Dépendances et versions.

Une seed est une valeur de départ qui fixe toutes les initialisations aléatoires du code, ce qui garantit que les résultats sont identiques à chaque exécution. Elle est fixée à 42 dans tous les notebooks, sauf le notebook 8 qui en teste cinq pour la robustesse inter-seed.

```
SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
```

V.B. Dataset : Mendeley BT-MRI

Les points suivants s'appliquent aux notebooks 1 et 3 qui utilisent tous les deux le dataset Mendeley BT-MRI.

V.B.1. Prétraitement et augmentation

Les transformations diffèrent entre train et test :

En train, une augmentation est appliquée pour éviter le sur-apprentissage. L'augmentation consiste à créer des variantes aléatoires des images (rotations, variations de luminosité, etc.) pour que le modèle ne mémorise pas les images d'entraînement et généralise mieux sur de nouvelles données. Les images sont aussi redimensionnées et normalisées, la normalisation ramène les valeurs des pixels dans un intervalle standard pour que les images soient comparables entre elles. Avec $\text{mean}=0.5$ (moyenne) et $\text{std}=0.5$ (écart-type), les valeurs des pixels sont ramenées dans l'intervalle $[-1, 1]$.

```
train_tf = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.Grayscale(num_output_channels=3),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
    transforms.Normalize([0.5]*3, [0.5]*3),
])
```

En test, seuls le redimensionnement et la normalisation sont appliqués, il n'y a pas d'augmentation.

V.B.2. Gestion du déséquilibre de classes

Les classes ne sont pas parfaitement équilibrées : notumor est sous-représenté avec 592 images en train contre 1 687 pour le gliome (avant split validation). Sans correction, le modèle verrait proportionnellement moins d'exemples de notumor pendant l'entraînement et aurait tendance à moins bien la reconnaître. Dans ces deux notebooks, j'utilise un `WeightedRandomSampler` pour rééquilibrer les batches, en donnant plus de chances aux classes sous-représentées d'être tirées.

```
counts = np.bincount([y for _, y in train_ds])

class_weights = 1.0 / counts
sample_weights = class_weights[[y for _, y in train_ds]]
sampler = WeightedRandomSampler(
    torch.tensor(sample_weights, dtype=torch.double),
    num_samples=len(sample_weights), replacement=True
)
```

V.C. Notebook 1 : CNN Baseline (ResNet18)

V.C.1. Architecture

Le CNN baseline est un ResNet18 dont les poids ont été appris sur ImageNet (une large base d'images naturelles). La tête de classification d'origine est conçue pour 1000 classes, je la remplace par un MLP à deux couches pour les 4 classes.

```
model = models.resnet18(weights="IMAGENET1K_V1")
model.fc = nn.Sequential(
    nn.Linear(512, 256),
    nn.ReLU(),
    nn.Dropout(0.4),
    nn.Linear(256, NUM_CLASSES)
)
```

Je réentraîne tout le réseau et pas seulement la tête. Avec peu d'images il vaut mieux geler les premières couches pour éviter le sur-apprentissage, mais avec 4 606 images en train on peut tout réentraîner.

[Approfondissement] ResNet18

Dans un réseau très profond (avec beaucoup de couches) le gradient s'atténue avant d'atteindre les premières couches, qui n'apprennent plus rien (22).

ResNet (15) règle ce problème avec des skip connections, qui créent un chemin direct pour le gradient, permettant aux premières couches de continuer à apprendre. La figure 13 montre ce principe sur une architecture DenseNet, où les connexions entre couches sont plus visibles. DenseNet relie les couches différemment de ResNet, mais crée lui aussi un chemin direct pour le gradient.

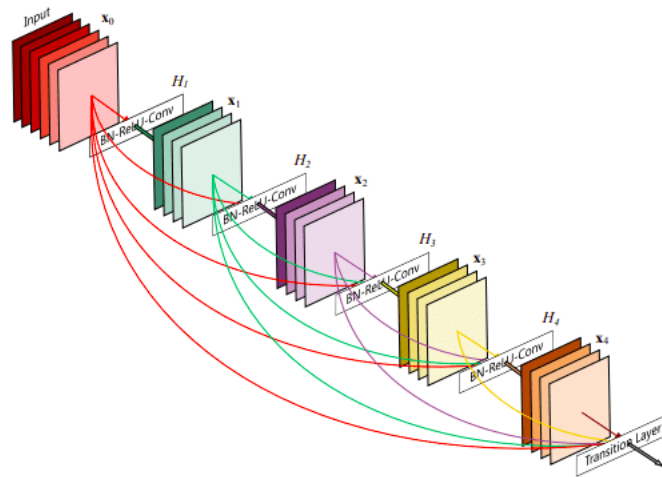


Figure 13 : Architecture DenseNet qui illustre le principe des skip connections. Tiré de (23).

Sans skip connections, la fonction de perte a de nombreux creux et pics. Le point le plus bas (celui que l'optimiseur cherche à atteindre, car il correspond aux poids qui minimisent l'erreur) est difficile à trouver. Avec des skip connections, la surface est lisse et l'optimiseur l'atteint plus facilement (figure 14).

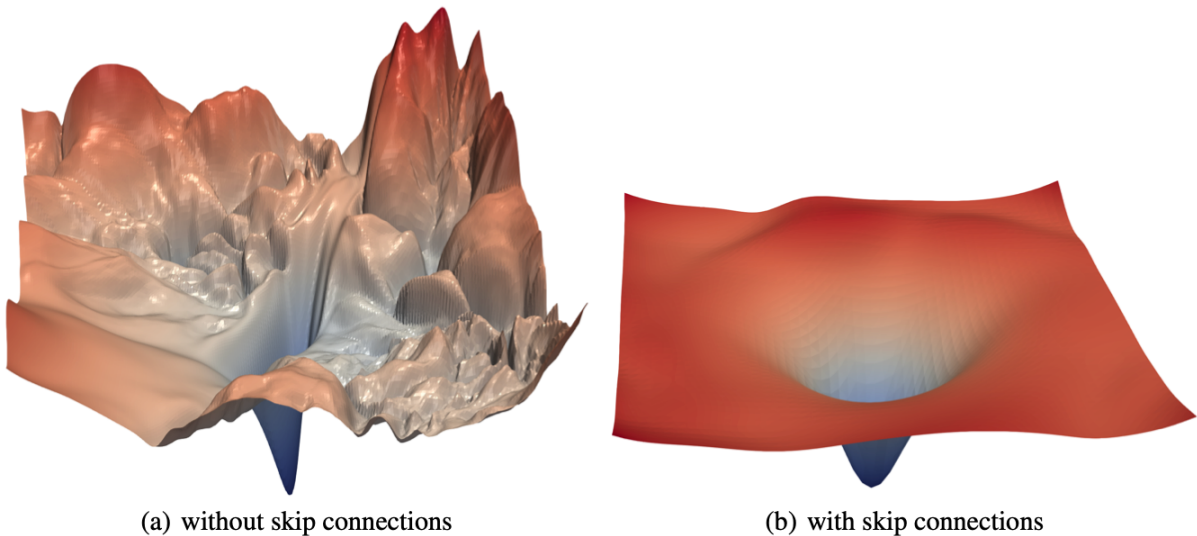


Figure 14 : Fonction de perte avec et sans skip connections. À gauche : sans skip connections. À droite : avec skip connections. Tiré de (24).

Il existe plusieurs variantes de ResNet avec plus ou moins de couches (ResNet18, ResNet50, ResNet101, etc.). J'utilise ResNet18 car il est suffisant pour 4 606 images.

V.C.2. Entraînement

J'utilise Adam (25) avec un learning rate de $1e-4$. Le scheduler StepLR divise ce taux par 2 tous les 5 epochs pour affiner progressivement les paramètres. La fonction de perte est la cross-entropie (standard pour la classification) : plus la prédiction est éloignée de la vraie classe, plus la pénalité est élevée.

```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LR)
scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.5)
```

Un early stopping (arrêt précoce) avec patience=3 arrête l'entraînement si l'AUC de validation ne progresse plus pendant 3 epochs consécutifs, pour éviter le sur-apprentissage. L'entraînement a duré 10 epochs, l'early stopping ne s'est pas déclenché.

V.D. Notebook 2 : Prédicteur d'âge (BrainAgeNet)

V.D.1. Extraction des coupes 2D depuis IXI

Les IRM du dataset IXI sont en volume 3D. Mon dataset principal étant en 2D, j'ai extrait 10 coupes autour du centre de chaque volume, ce qui permet aussi d'avoir plus de données d'entraînement.

```
y_center = vol.shape[1] // 2
y_start = y_center - N_SLICES_PER_VOLUME // 2
y_end = y_center + N_SLICES_PER_VOLUME // 2

for y_idx in range(y_start, y_end):
    slc = vol[:, y_idx, :]
    slc = normalize_slice(slc)
    slc_img = Image.fromarray(slc).convert("L")
```

Le split train/val est fait au niveau des sujets et non des coupes, pour éviter qu'un même sujet se retrouve à la fois en train et en test. Je n'ai pas fait de jeu de test pour garder suffisamment d'images pour l'entraînement (seulement 588 sujets).

```
subject_ids = list(set([os.path.basename(r[0]).split("_")[0] for r in records]))
train_ids, val_ids = train_test_split(subject_ids, test_size=0.2,
random_state=SEED)
```

V.D.2. Architecture BrainAgeNet

J'utilise EfficientNet-B0 comme encodeur (la partie du réseau qui extrait les features visuelles) dont les poids ont été appris sur ImageNet. La tête de classification d'origine est conçue pour 1000 classes, je la remplace par une tête de régression qui prédit un nombre entre 0 et 1 plutôt qu'une classe, correspondant à un âge entre 20 et 86 ans (minimum et maximum du dataset). Pour récupérer l'âge en années : $\text{âge (ans)} = \text{âge (normalisé)} \times (86 - 20) + 20$.

```
class BrainAgeNet(nn.Module):
    def __init__(self):
        super().__init__()
        backbone = efficientnet_b0(weights=EfficientNet_B0_Weights.IMAGENET1K_V1)
        in_features = backbone.classifier[1].in_features
        backbone.classifier = nn.Identity()
        self.encoder = backbone

        self.age_head = nn.Sequential(
            nn.Linear(in_features, 256),
            nn.BatchNorm1d(256),
            nn.ReLU(),
            nn.Dropout(0.4),
            nn.Linear(256, 64),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(64, 1),
            nn.Sigmoid()
        )
```

[Approfondissement] EfficientNet-B0

EfficientNet ajuste simultanément les CNN selon trois dimensions : profondeur, largeur et résolution d'entrée. Les augmenter proportionnellement donne de meilleurs résultats que de les augmenter séparément, comme démontré par Tan & Le (26).

J'ai utilisé EfficientNet-B0, la version la plus compacte avec 5.3 millions de paramètres. Avec seulement 588 sujets dans le dataset IXL, un modèle plus grand aurait sur-appri.

V.D.3. Fonction de perte et optimiseur

La fonction de perte est une MSE pondérée (Weighted Mean Squared Error) : la pondération donne un poids de 1 aux âges proches du milieu de l'intervalle (0.5, soit 53 ans) et jusqu'à 2 aux âges extrêmes, ce qui force le modèle à ne pas ignorer les sujets très jeunes ou très âgés.

```
def weighted_mse_loss(pred, target):  
    weights = 1.0 + 2.0 * torch.abs(target - 0.5)  
    return (weights * (pred - target) ** 2).mean()
```

L'optimiseur est AdamW (27), une variante d'Adam qui ajoute une pénalité sur les poids trop grands pour éviter le sur-apprentissage. Le scheduler est un CosineAnnealingWarmRestarts, qui réduit le learning rate progressivement puis le réinitialise périodiquement.

V.D.4. Application au dataset tumoral

Une fois entraîné sur le dataset IXI, BrainAgeNet est appliqué à toutes les images du dataset tumoral. Les prédictions sont sauvegardées dans predicted_clinical.pkl, un fichier qui associe chaque image à son âge prédit.

Ce fichier est chargé dans les notebooks suivants pour fournir la feature clinique. Le modèle a été entraîné uniquement sur des cerveaux sains, les prédictions sur des IRM tumorales restent donc approximatives.

V.E. Notebook 3 : BNN

V.E.1. Architecture BNNMultimodalNet

La branche visuelle charge les poids du CNN baseline entraîné dans le notebook 1. Seules les deux dernières couches (layer3 et layer4) restent entraînaibles, les couches initiales (conv1, layer1, layer2) sont gelées. Ces couches initiales détectent des features génériques comme les contours et les textures, déjà apprises sur ImageNet. Les réentraîner aurait créé un risque de sur-apprentissage.

```

class BNNMultimodalNet(nn.Module):
    def __init__(self, num_classes=4, clinical_dim=1,
                  cnn_ckpt=None, prior_mu=0, prior_sigma=0.1):
        super().__init__()

        backbone = models.resnet18(weights="IMAGENET1K_V1")
        backbone.fc = nn.Identity()
        self.cnn = backbone

        for name, param in self.cnn.named_parameters():
            if "layer4" in name or "layer3" in name:
                param.requires_grad = True
            else:
                param.requires_grad = False

        self.clinical_mlp = nn.Sequential(
            nn.Linear(clinical_dim, 64),
            nn.ReLU(),
            nn.Linear(64, 64),
            nn.ReLU(),
            nn.Linear(64, 32),
            nn.ReLU(),
        )

        self.bnn_head = nn.Sequential(
            bnn.BayesLinear(prior_mu=prior_mu, prior_sigma=prior_sigma,
                           in_features=512 + 32, out_features=256),
            nn.ReLU(),
            bnn.BayesLinear(prior_mu=prior_mu, prior_sigma=prior_sigma,
                           in_features=256, out_features=128),
            nn.ReLU(),
            bnn.BayesLinear(prior_mu=prior_mu, prior_sigma=prior_sigma,
                           in_features=128, out_features=num_classes),
        )

    def forward(self, img, clinical):
        img_feat = self.cnn(img)
        clinical_feat = self.clinical_mlp(clinical)
        fused = torch.cat([img_feat,
                           clinical_feat], dim=1)

        return self.bnn_head(fused)

```

torchbnn permet de rendre tout le réseau bayésien, pas seulement la tête. J'ai choisi de limiter la partie bayésienne à la tête car entraîner un ResNet entièrement bayésien aurait été trop long. De plus, Kristiadi et al. (28) montrent qu'une tête bayésienne suffit à améliorer significativement la calibration.

V.E.2. La divergence KL

```
def kl_loss(model):
    kl = torch.tensor(0.0, device=device)
    for m in model.modules():
        if isinstance(m, bnn.BayesLinear):
            mu = m.weight_mu
            log_sigma = m.weight_log_sigma
            sigma = torch.exp(log_sigma)
            prior_mu = torch.tensor(m.prior_mu, device=device)
            prior_sig = torch.tensor(m.prior_sigma, device=device)
            kl += 0.5 * torch.mean(
                ((mu - prior_mu) ** 2 + sigma ** 2) / (prior_sig ** 2)
                - 1
                + 2 * torch.log(prior_sig)
                - 2 * log_sigma
            )
    return kl
```

Cette formule calcule la divergence KL dans le cas de deux distributions gaussiennes, c'est ce que torchbnn implémente dans ses couches BayesLinear.

La loss totale combine la cross-entropie et la pénalité KL pondérée à 0.01 comme présenté en sous-section II.B.6.

```
loss = ce + KL_WEIGHT * kl
```

V.E.3. Optimiseur et scheduler

Seuls les paramètres de layer3, layer4, du MLP clinique et de la tête BNN sont entraînaibles et passés à l'optimiseur.

```
trainable = [p for p in model.parameters() if p.requires_grad]
```

Le learning rate initial de 1e-3 est plus élevé que le 1e-4 du CNN baseline car la tête BNN est initialisée aléatoirement, contrairement au ResNet18 dont les poids ont déjà été appris sur ImageNet.

```
optimizer = torch.optim.Adam(trainable, lr=1e-3)
scheduler = torch.optim.lr_scheduler.CosineAnnealingWarmRestarts(
    optimizer, T_0=5, T_mult=2
)
```

J'ai mis un early stopping avec patience=3, comme pour le CNN baseline. Les 15 epochs ont été effectuées, l'early stopping ne s'est pas déclenché.

V.E.4. Inférence Monte-Carlo

Pendant l'inférence, j'appelle `model.eval()` :

- Pour `torchbnn`, cela suffit car les couches `BayesLinear` continuent de tirer des poids différents à chaque passe.
- Pour MC Dropout en revanche, `model.eval()` désactive le dropout par défaut et les prédictions deviennent déterministes. Il faut donc appeler `enable_mc_dropout()` après `model.eval()` pour forcer le dropout à rester actif, ce qui est fait dans le notebook 6.

`torch.no_grad()` désactive le calcul des gradients, ce qui accélère les calculs. Pour chaque image, 50 prédictions sont calculées (courbe de convergence, VI.D) : la moyenne donne la prédiction finale et l'écart-type donne l'incertitude.

```
def bayesian_inference(loader, model, n_passes=MC_PASSES):
    model.eval()
    all_y, all_mean, all_std = [], [], []
    with torch.no_grad():
        for img, clin, y in loader:
            img, clin = img.to(device), clin.to(device)
            passes = torch.stack([
                torch.softmax(model(img, clin), dim=1)
                for _ in range(n_passes)
            ])
            all_y.extend(y.numpy())
            all_mean.extend(passes.mean(dim=0).cpu().numpy())
            all_std.extend(passes.std(dim=0).cpu().numpy())
    sigma = np.array(all_std).max(axis=1)
    return np.array(all_y), np.array(all_mean), sigma
```

V.E.5. BNN Image Only

Le notebook entraîne deux variantes : le BNN multimodal décrit ci-dessus, et un BNN Image Only identique mais sans la branche clinique. Le multimodal sert de comparaison pour mesurer ce que l'âge prédit apporte réellement.

V.F. Notebook 4 : Analyse de l'incertitude et calibration

V.F.1. Incertitude par classe

Pour voir si certaines classes sont plus difficiles à prédire que d'autres, je prends 30 images aléatoires par classe. Au-delà de 30 images, l'estimation se stabilise (VI.F.1). L'âge est fixé au point du milieu de l'intervalle (0.5, soit 53 ans) pour toutes les images pour que la comparaison ne dépende que du type de tumeur. Ici je mesure σ sur la classe réelle de l'image, pour comparer la difficulté entre classes.

```
CLIN_MED = np.array([[0.5]], dtype=np.float32)

for p in sampled:
    img_t      = load_image(p)
    _, std_p    = predict_bnn(img_t, CLIN_MED)
    sigma_per_class[cls].append(std_p[0, true_idx])
```

V.F.2. Calibration (ECE)

Je calcule l'ECE sur l'ensemble du jeu de test. Je divise les prédictions en 10 groupes selon leur niveau de confiance (0-10 %, 10-20 %, etc.). La confiance représente la moyenne des probabilités données à la classe prédite sur les 50 passes MC. Une prédiction est soit juste, soit fausse, on ne peut donc pas dire individuellement si la probabilité est correcte. Par groupe, en revanche, on peut évaluer la calibration : si dans le groupe 90-100 % le modèle affiche en moyenne 94 % de confiance mais a raison 96 % du temps, il a en fait raison plus souvent que ce qu'il affiche. On calcule ces écarts pour chaque groupe. L'ECE est la moyenne de ces écarts, pondérée par le nombre de prédictions dans chaque groupe.

```
M = 10
for i in range(M):
    mask = (all_confs >= bins[i]) & (all_confs < bins[i+1])
    bin_accs[i] = all_corrects[mask].mean()
    bin_confs[i] = all_confs[mask].mean()
ECE = np.sum(bin_counts / len(all_confs) * np.abs(bin_accs - bin_confs))
```

V.G. Notebook 5 : Grad-CAM et explicabilité visuelle

V.G.1. Implémentation de GradCAM

La classe GradCAM utilise des hooks (fonctions s'exécutant automatiquement à certains moments de la propagation avant et arrière) : un pendant la propagation avant pour capturer les activations de la dernière couche convolutive, et un pendant la rétropropagation pour capturer les gradients.

```
class GradCAM:
    def __init__(self, model, target_layer):
        self.model = model
        self.gradients = self.activations = None
        target_layer.register_forward_hook(self._save_activation)
        target_layer.register_full_backward_hook(self._save_gradient)

    def _save_activation(self, module, input, output):
        self.activations = output.detach()

    def _save_gradient(self, module, grad_input, grad_output):
        self.gradients = grad_output[0].detach()

    def compute(self, img_tensor, clinical_tensor, class_idx):
        self.model.eval()
        img_tensor.requires_grad_(True)
        logits = self.model(img_tensor, clinical_tensor)
        self.model.zero_grad()
        logits[0, class_idx].backward()
        weights = self.gradients.mean(dim=[2, 3], keepdim=True)
        cam = (weights * self.activations).sum(dim=1, keepdim=True)
        cam = torch.relu(cam).squeeze().cpu().numpy()
        cam = (cam - cam.min()) / (cam.max() - cam.min() + 1e-8)
        return cam

gradcam = GradCAM(model, model.cnn.layer4)
```

J'utilise `register_full_backward_hook` plutôt que `register_backward_hook`, qui est déprécié depuis PyTorch 1.8 et risque de produire des gradients incorrects.

V.G.2. Grad-CAM avec 50 passes MC

50 cartes Grad-CAM sont calculées, une par passe MC. Les poids de la tête du BNN sont différents à chaque passe (tirés aléatoirement depuis leurs distributions). Comme seule la tête est bayésienne et que layer4 appartient à la partie déterministe du ResNet, les activations de layer4 sont identiques à chaque passe, ce sont uniquement les gradients qui varient, via la rétropropagation depuis la tête BNN stochastique. La carte d'incertitude spatiale mesure donc uniquement la variabilité introduite par la tête bayésienne, pas par le ResNet.

```
def compute_mc_gradcam(gradcam_obj, img_tensor, clinical_tensor,
                      class_idx, n_passes=50):
    cams = []
    for _ in range(n_passes):
        cam = gradcam_obj.compute(img_tensor.clone(), clinical_tensor, class_idx)
        cams.append(cam)
    cams = np.stack(cams, axis=0)
    return cams.mean(axis=0), cams.std(axis=0)
```

La carte std (écart-type pixel par pixel) donne l'incertitude spatiale : les zones avec un fort écart-type sont celles où l'attention du modèle varie le plus entre les passes.

Cette carte capture donc uniquement l'incertitude introduite par la tête bayésienne. Un réseau entièrement bayésien donnerait une carte plus complète, mais le temps d'entraînement serait bien plus long.

V.H. Notebook 6 : Variante MC Dropout

Pour comparer torchbnn avec MC Dropout dans les mêmes conditions, j'entraîne un modèle avec la même architecture que le BNN Image Only, sauf que la tête bayésienne est remplacée par un MLP classique avec du dropout maintenu actif à l'inférence. Tout le reste est identique : même split train/test, même seed, même optimiseur, même early stopping avec patience=3. L'entraînement s'est arrêté à l'epoch 8, l'early stopping s'est déclenché.

J'entraîne trois variantes avec des taux de dropout différents (0.2, 0.3, 0.5) car l'incertitude estimée par MC Dropout dépend fortement de ce paramètre. Le taux 0.3 est utilisé comme référence pour la comparaison principale.

```

class MCDropoutImageOnly(nn.Module):
    def __init__(self, num_classes=4, dropout=0.3):
        super().__init__()
        backbone = models.resnet18(weights="IMAGENET1K_V1")
        backbone.fc = nn.Identity()
        self.cnn = backbone
        for name, param in self.cnn.named_parameters():
            if "layer4" in name or "layer3" in name:
                param.requires_grad = True
            else:
                param.requires_grad = False
        self.head = nn.Sequential(
            nn.Dropout(dropout),
            nn.Linear(512, 256), nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(256, num_classes)
        )

    def forward(self, img, clin=None):
        return self.head(self.cnn(img))

    def enable_mc_dropout(self):
        for m in self.modules():
            if isinstance(m, nn.Dropout):
                m.train()

```

V.I. Notebook 7 : Validation externe Br35H

V.I.1. Protocole

J'évalue les trois modèles (CNN baseline, BNN torchbnn, BNN MC Dropout) sur Br35H sans aucun réentraînement. C'est ce qu'on appelle une évaluation zero-shot : les modèles n'ont jamais vu ces images pendant l'entraînement.

Comme Br35H n'a que deux classes (tumeur / pas tumeur) et que les modèles en ont 4, on convertit les prédictions : la probabilité d'avoir une tumeur est $1 - P(\text{notumor})$.

```

def to_binary(probs):
    return 1 - probs[NOTUMOR_IDX]

```

V.I.2. Détection du changement (shift) de distribution

On compare l'incertitude σ des prédictions sur le Mendeley BT-MRI avec celle sur Br35H. Si le modèle détecte bien qu'il est sur des données nouvelles, son σ devrait être plus élevé sur Br35H. Je vérifie que cette différence n'est pas due au hasard avec un test de Mann-Whitney, un test qui compare deux distributions sans hypothèse sur leur forme. Si la p-value est inférieure à 0.05, la différence est réelle et non accidentelle.

V.I.3. Validation quantitative des Grad-CAM

Les cartes Grad-CAM sont évaluées sur 200 images annotées de Br35H grâce aux masques de segmentation. Trois métriques sont calculées :

- **L'IOU** mesure le chevauchement entre la zone activée par la carte et le masque : 0 = aucun chevauchement, 1 = superposition parfaite.
- **Le pointing game** vérifie si le pixel le plus activé de la carte tombe dans le masque. C'est une métrique binaire par image : 1 si oui, 0 sinon.
- **L'energy ratio** mesure quelle proportion de l'activation totale de la carte se trouve dans le masque.

```
def compute_metrics(cam, mask):
    cam_bin = (cam >= 0.5).astype(float)
    intersection = (cam_bin * mask).sum()
    union = ((cam_bin + mask) > 0).sum()
    iou = intersection / union if union > 0 else 0

    max_idx = np.unravel_index(cam.argmax(), cam.shape)
    pg = float(mask[max_idx] > 0)

    er = (cam * mask).sum() / cam.sum() if cam.sum() > 0 else 0

    return iou, pg, er
```

V.J. Notebook 8 : Robustesse inter-seed

Ce notebook réentraîne le BNN Image Only (torchbnn) et MC Dropout (taux 0.3) sur cinq seeds (0, 1, 2, 3 et 42), pour vérifier que les résultats ne dépendent pas de la seed choisie.

Les deux modèles ne convergent pas de la même façon : MC Dropout atteint son maximum dès l'epoch 5 puis baisse un peu, torchbnn met plus de temps et plafonne vers l'epoch 14. L'early stopping (patience=3) pouvait couper torchbnn trop tôt sur certaines seeds, pendant un creux d'AUC après un redémarrage du scheduler. C'est arrivé sur la seed 3 (AUC 0.694). J'ai retiré l'early stopping et gardé le meilleur modèle sur la validation, sur un nombre d'epochs fixe (15). La seed 3 atteint alors 0.995, et la seed 42 ne bouge pas : son early stopping ne s'était jamais déclenché, donc les chiffres du mémoire restent les mêmes.

```
def train_bnn(seed, train_loader, val_loader):
    random.seed(seed); np.random.seed(seed); torch.manual_seed(seed)
    model = BNNImageOnly(cnn_ckpt=CNN_CKPT).to(device)
    trainable = [p for p in model.parameters() if p.requires_grad]
    optimizer = torch.optim.Adam(trainable, lr=LR)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingWarmRestarts(optimizer,
T_0=5, T_mult=2)
    ce_loss = nn.CrossEntropyLoss()

    best_auc, best_state = -1.0, None
    for epoch in range(1, EPOCHS + 1):
        model.train()
        for img, clin, y in train_loader:
            img, y = img.to(device), y.to(device)
            optimizer.zero_grad()
            loss = ce_loss(model(img), y) + KL_WEIGHT * kl_loss(model)
            loss.backward(); optimizer.step()
        scheduler.step()

        _, _, _, _, auc_v, _ = mc_inference(val_loader, model)
        if auc_v > best_auc:
            best_auc = auc_v
            best_state = {k: v.cpu().clone() for k, v in
model.state_dict().items()}

    model.load_state_dict(best_state)
    return model
```

Pour chaque seed, je recalcule l'AUC, l'ECE, le σ médian in-distribution sur le jeu de test du Mendeley BT-MRI, le ratio $\sigma_{\text{incorrect}} / \sigma_{\text{correct}}$, et les métriques de triage : accuracy globale, accuracy sur la moitié la plus confiante, et pourcentage des erreurs couvertes en renvoyant 20 % des prédictions les plus incertaines.

V.K. Notebook 9 : Analyses complémentaires

Le notebook 9 regroupe les analyses supplémentaires réalisées pour valider et enrichir les résultats :

- Scatter plot BrainAgeNet avec KDE et R^2 ,
- Bootstrap sur les ratios σ par classe,
- Stabilité de l'estimation σ selon le nombre d'images,
- Comparaison des architectures ResNet18, ResNet34 et ResNet50,
- Comparaison des learning rates,
- Présentation visuelle des trois datasets.

VI. Résultats

Les résultats sont évalués sur le jeu de test du dataset Mendeley BT-MRI (dossier Test/, 1 149 images, soit 20 % du dataset total), sauf pour BrainAgeNet qui est évalué sur le jeu de validation du dataset IXI.

VI.A. CNN Baseline

Le CNN baseline atteint une AUC macro OvR (One vs Rest) de 0.998, une balanced accuracy (moyenne non pondérée de la précision par classe) de 0.966 et une ECE de 0.0160 sur le jeu de test.

L'AUC macro OvR est calculée pour chaque classe de manière binaire : gliome ou pas gliome, méningiome ou pas méningiome, etc. Macro signifie que la moyenne ne pondère pas les classes en fonction du nombre d'images de chacune.

Je l'utilise comme baseline pour comparer avec le BNN.

VI.A.1. Choix du modèle ResNet

J'ai comparé trois variantes de ResNet pour justifier le choix du ResNet18.

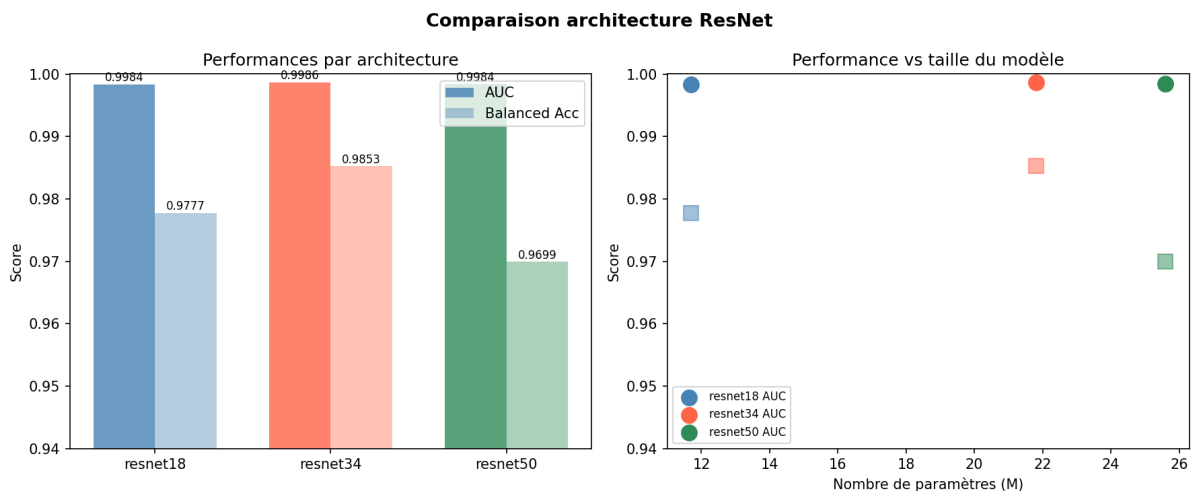


Figure 15 : Comparaison des architectures ResNet18, ResNet34 et ResNet50. À gauche : AUC et balanced accuracy sur le jeu de test. À droite : performance en fonction du nombre de paramètres du modèle.

ResNet18 obtient une AUC comparable aux variantes plus profondes avec deux fois moins de paramètres (figure 15). ResNet34 fait un peu mieux en balanced accuracy (0.9853 contre 0.9777), mais l'écart est faible, alors j'ai préféré utiliser le modèle le plus léger.

VI.A.2. Choix du learning rate

J'ai testé cinq learning rates pour le CNN baseline.

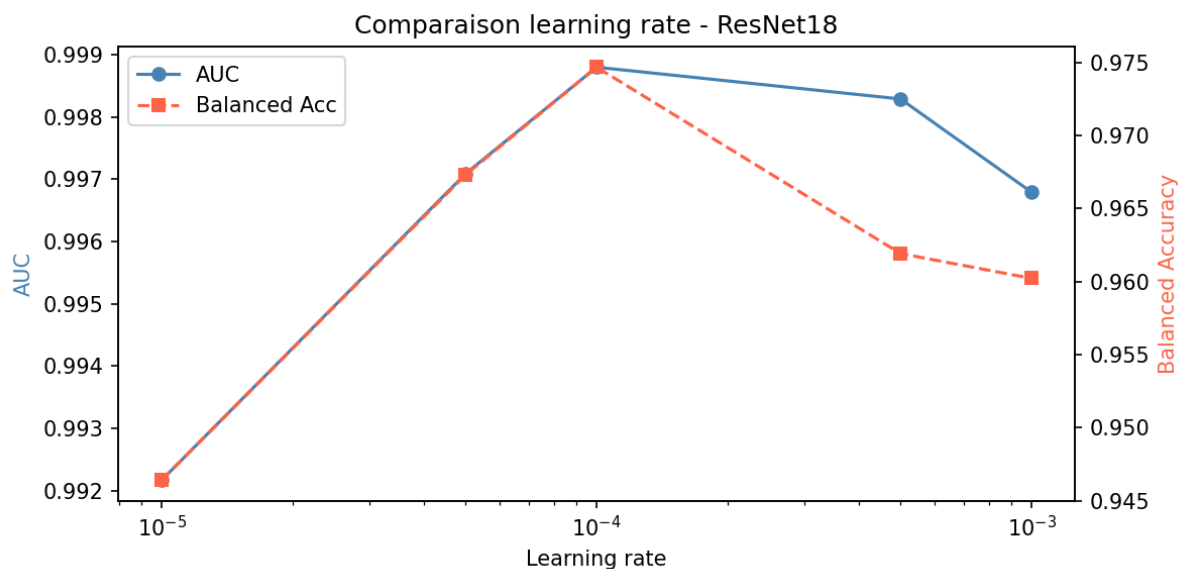


Figure 16 : Performances du CNN baseline selon le learning rate.

Le learning rate de $1e-4$ donne les meilleures performances (figure 16). En dessous de $1e-4$, l'AUC finale est moins bonne. Au-dessus, les performances baissent aussi.

Les figures 15 et 16 viennent de runs séparés du modèle final. La balanced accuracy n'est donc pas exactement celle de mon baseline (0.966), mais l'AUC reste stable.

VI.B. Prédicteur d'âge (BrainAgeNet)

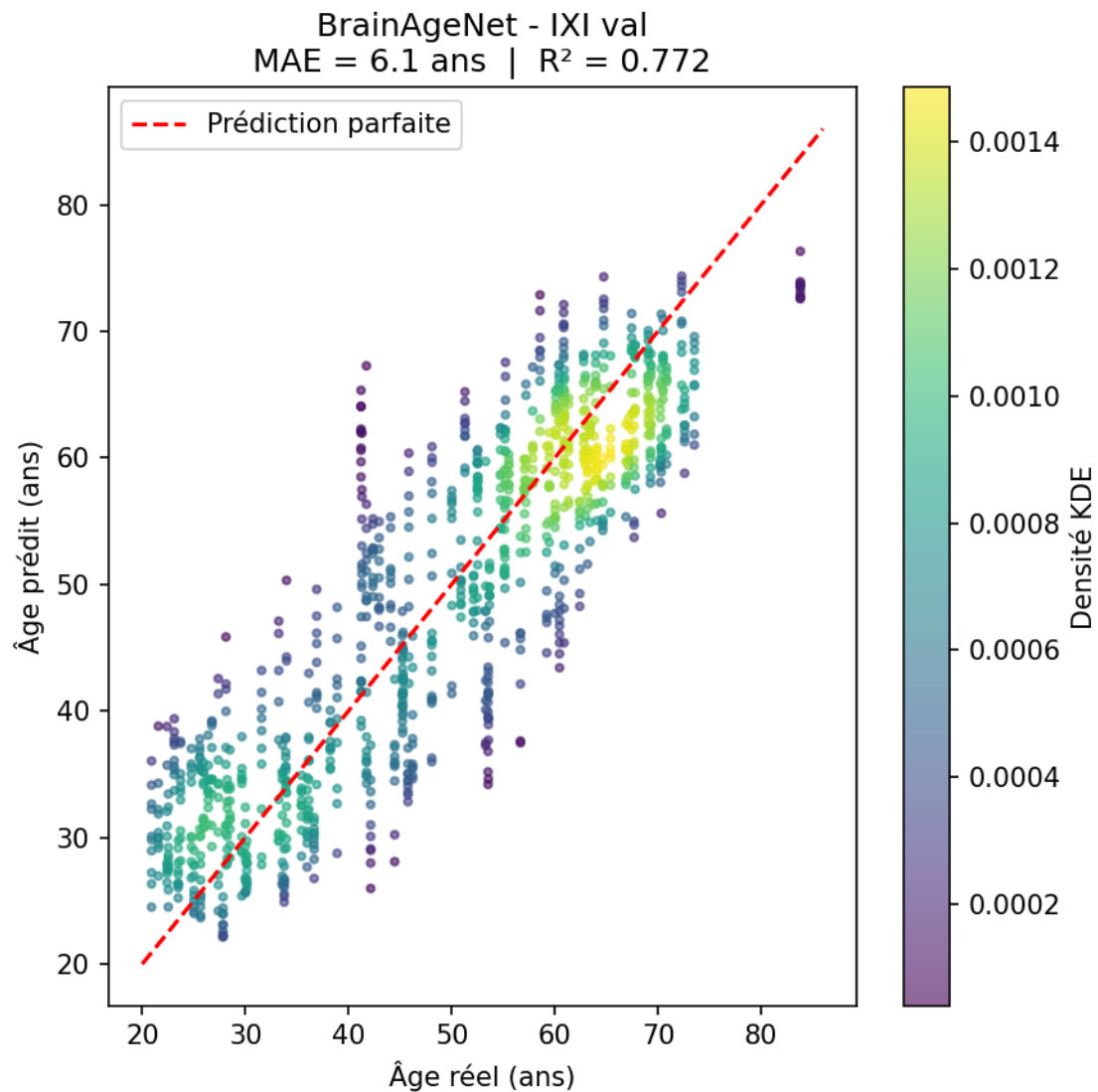


Figure 17 : Scatter plot âge prédit vs âge réel par BrainAgeNet sur le set de validation IXI. La couleur représente la densité locale des points (KDE, Kernel Density Estimation) : les zones claires concentrent le plus de prédictions.

BrainAgeNet atteint un MAE de 6.1 ans et un R^2 de 0.772 sur le set de validation (figure 17). Le MAE mesure l'écart moyen entre l'âge prédit et l'âge réel. Le R^2 mesure à quel point les prédictions suivent la droite de prédiction parfaite : 0 signifie que le modèle ne prédit rien, 1 signifie une prédiction parfaite. Ces estimations sont utilisées comme feature clinique dans le BNN.

VI.C. Convergence du BNN

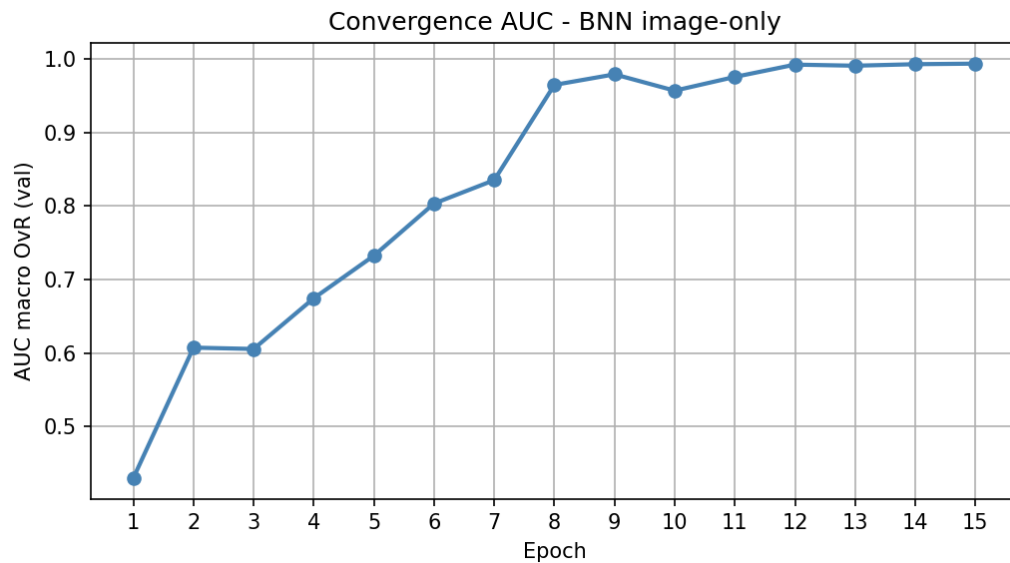


Figure 18 : Courbe AUC pendant l'entraînement du BNN sur le set de validation.

L'AUC atteint 0.993 sur le set de validation (0.995 sur le test) (figure 18).

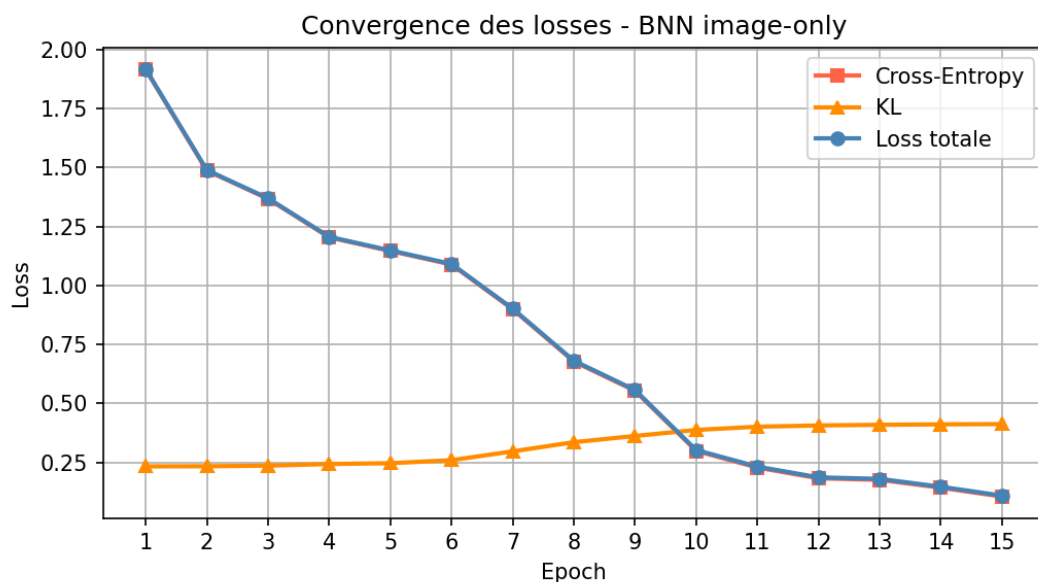


Figure 19 : Courbes de loss, cross-entropie et divergence KL pendant l'entraînement du BNN.

La loss totale et la cross-entropie convergent bien vers une valeur basse, proche de 0 (figure 19). La divergence KL, elle, augmente jusqu'à atteindre un plateau (le modèle s'éloigne petit à petit du prior en apprenant).

VI.D. Nombre de passes Monte-Carlo

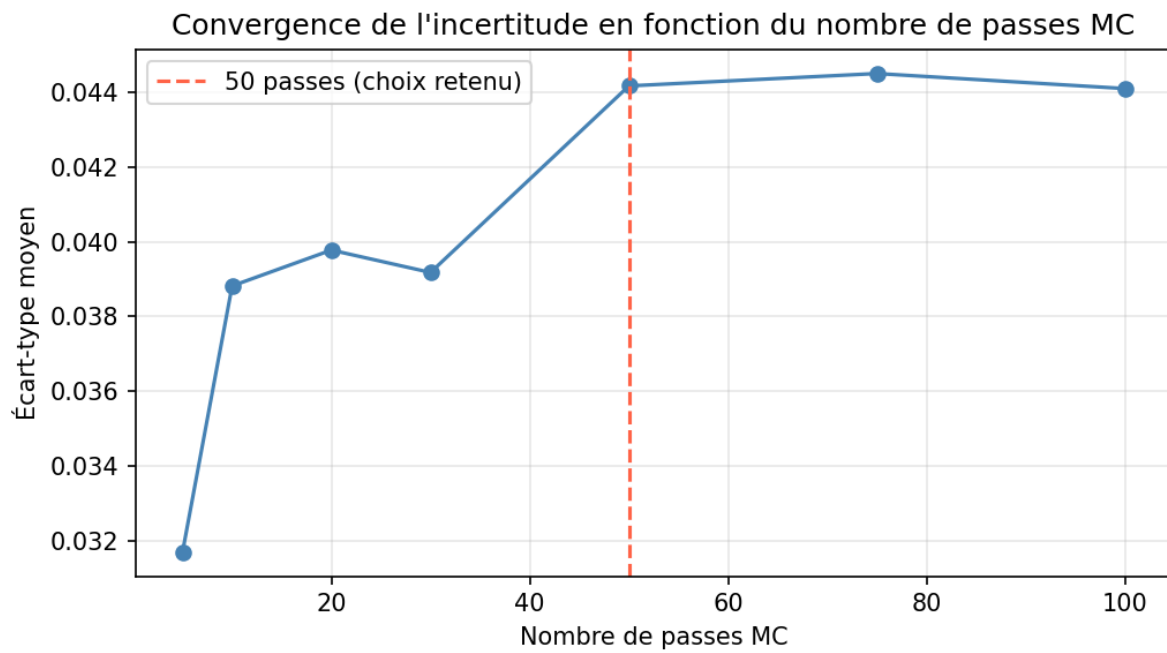


Figure 20 : Convergence de l'incertitude épistémique en fonction du nombre de passes MC.

J'ai testé différents nombres de passes MC (5, 10, 20, 30, 50, 75, 100) sur 200 images de test. L'écart-type moyen se stabilise à partir de 50 passes (figure 20), j'ai donc choisi 50.

VI.E. Matrice de confusion

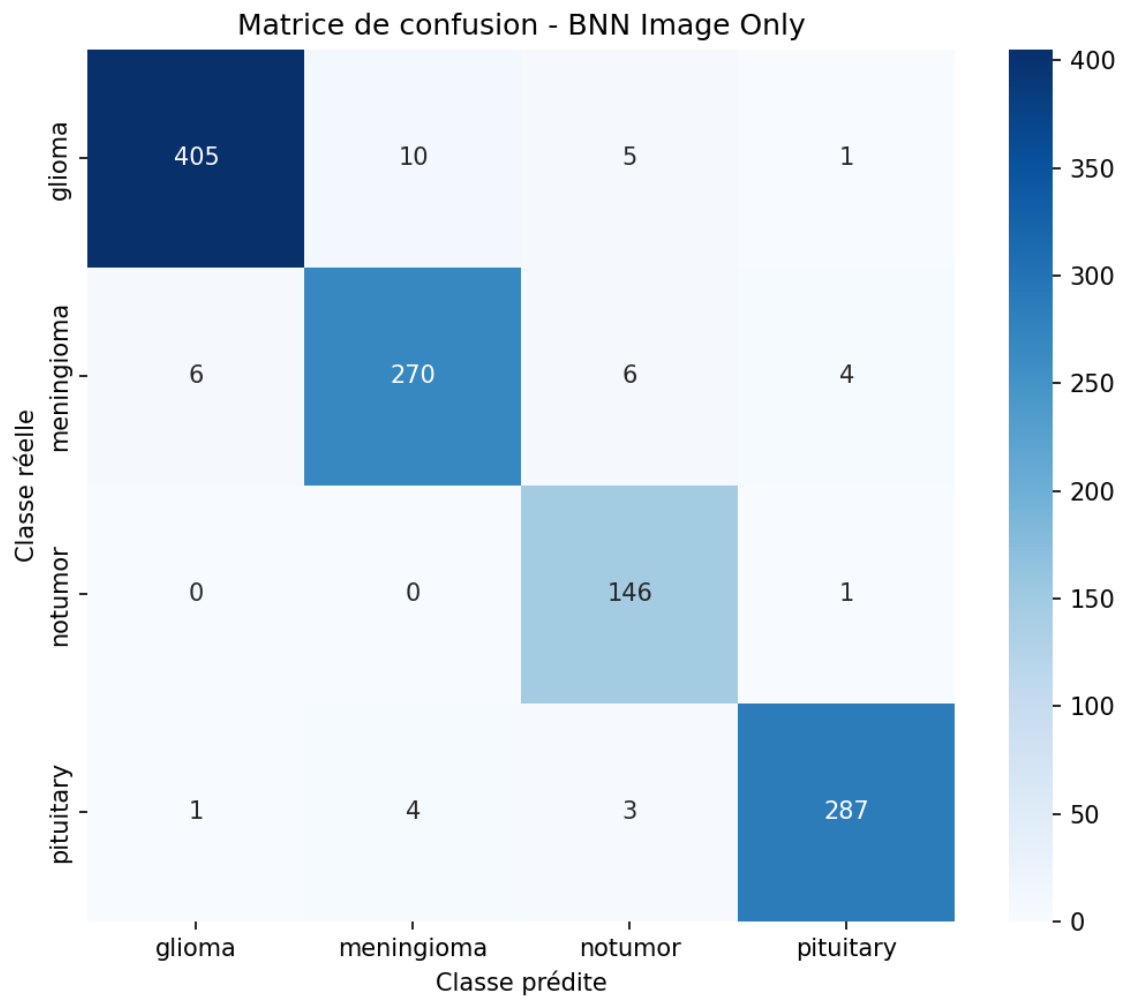


Figure 21 : Matrice de confusion du BNN.

La classe notumor est presque parfaitement classifiée (146/147, 1 erreur) et la pituitaire presque parfaite (287/295, 8 erreurs). Le gliome (405/421) et le méningiome (270/286) sont les classes les plus difficiles, avec 16 erreurs chacune (figure 21).

VI.F. Incertitude par classe

Rappel : ici σ est pris sur la classe réelle de l'image, alors que le ratio et le triage utilisent le σ maximal sur les classes.

VI.F.1. Distribution de l'incertitude

Cette analyse porte sur 30 images aléatoires par classe. Au-delà de ce seuil, l'estimation se stabilise pour toutes les classes, donc 30 images suffisent (figure 22).

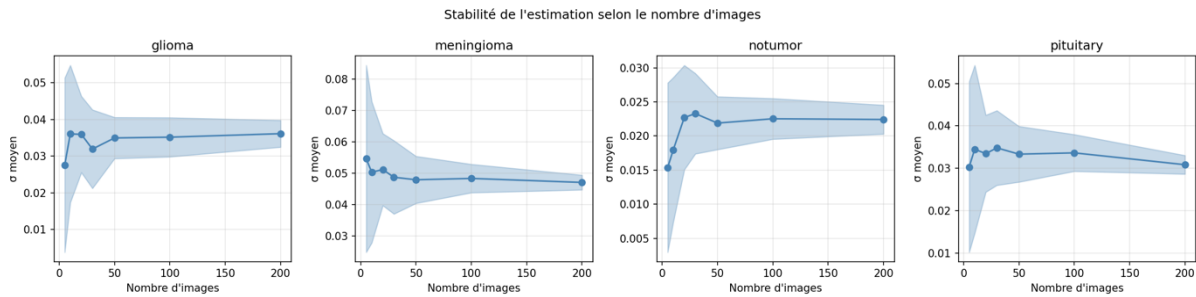


Figure 22 : Stabilité de l'estimation de l'incertitude selon le nombre d'images par classe. À partir de 30 images, l'écart-type de l'estimation se stabilise pour toutes les classes.

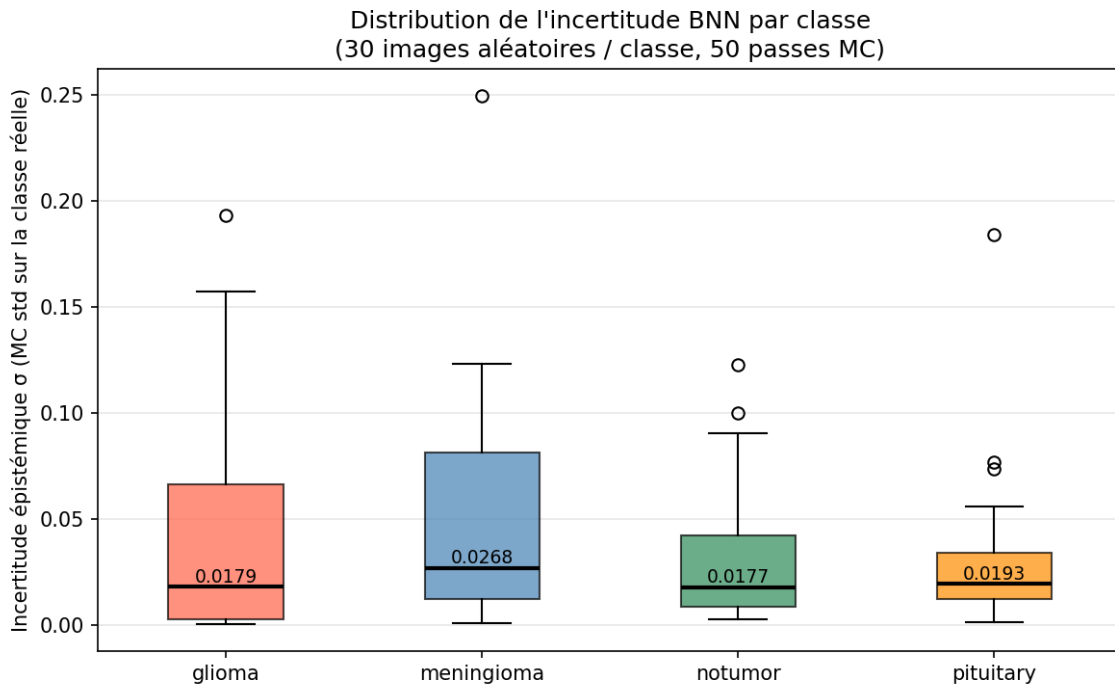


Figure 23 : Distribution de l'incertitude épistémique du BNN par classe. Chaque boîte représente la distribution du σ sur 30 images aléatoires. La ligne centrale est la médiane, les bords de la boîte le premier et troisième quartile, les traits s'étendent jusqu'à 1.5 fois l'écart interquartile, et les points au-delà sont des valeurs extrêmes.

La classe méningiome se détache avec une incertitude médiane plus élevée que les autres classes (0.0268 contre 0.0177 à 0.0193 pour les autres), ce qui se traduit par une plus grande difficulté du modèle à identifier cette classe. Le gliome présente aussi plusieurs valeurs extrêmes, ce qui confirme que certaines images sont plus difficiles que d'autres pour cette classe (figure 23).

VI.F.2. Ratio $\sigma_{\text{incorrect}} / \sigma_{\text{correct}}$

Le calcul du ratio est fait sur l'ensemble des prédictions du jeu de test (1 149 images), contrairement à l'analyse précédente qui se limite à 30 images par classe. Quand le modèle se trompe, son écart-type, noté σ (sigma), des 50 prédictions MC est bien plus élevé : $\sigma_{\text{incorrect}} = 0.1467$ contre $\sigma_{\text{correct}} = 0.0304$, soit 4.83 fois plus élevé. Décomposé par classe :

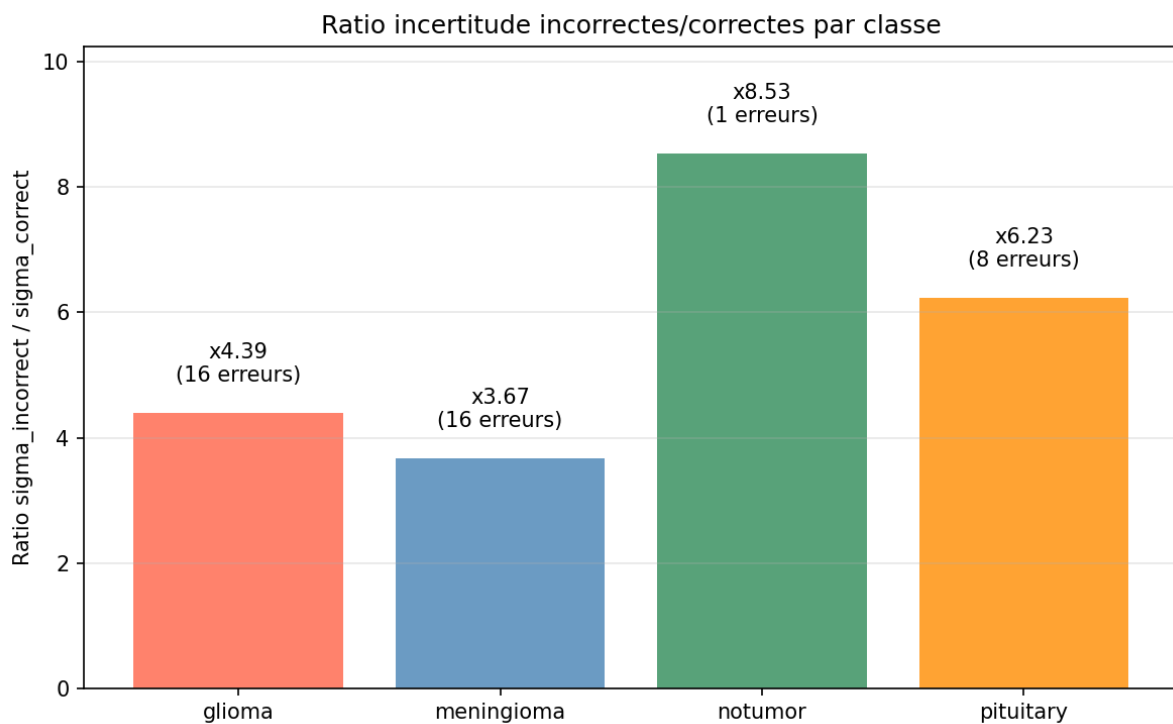


Figure 24 : Ratio $\sigma_{\text{incorrect}} / \sigma_{\text{correct}}$ par classe.

La classe notumor a un ratio de 8.53 mais sur une seule erreur (le modèle ne se trompe presque jamais sur cette classe), et la classe pituitaire 6.23 sur 8 erreurs, ce qui reste peu représentatif. Pour les classes avec suffisamment d'erreurs, le gliome (16 erreurs) et le méningiome (16 erreurs), le ratio est de 4.39 et 3.67 (figure 24).

VI.F.3. Confirmation des ratios par Bootstrap

Le bootstrap consiste à répéter le calcul du ratio sur des sous-échantillons aléatoires du jeu de test pour estimer la variabilité du résultat. L'intervalle de confiance à 95 % (IC95) donne la plage dans laquelle le vrai ratio se situe avec 95 % de probabilité.

Le bootstrap confirme ces ratios sur 25 % du jeu de test (1 000 itérations) (figure 25) :

- **Gliome** : médiane **4.07**, IC95 = **[0.88, 6.87]**
- **Méningiome** : médiane **3.16**, IC95 = **[1.39, 5.33]**
- **Notumor** : médiane **5.44**, IC95 = **[3.84, 7.96]**
- **Pituitaire** : médiane **3.73**, IC95 = **[2.94, 5.01]**

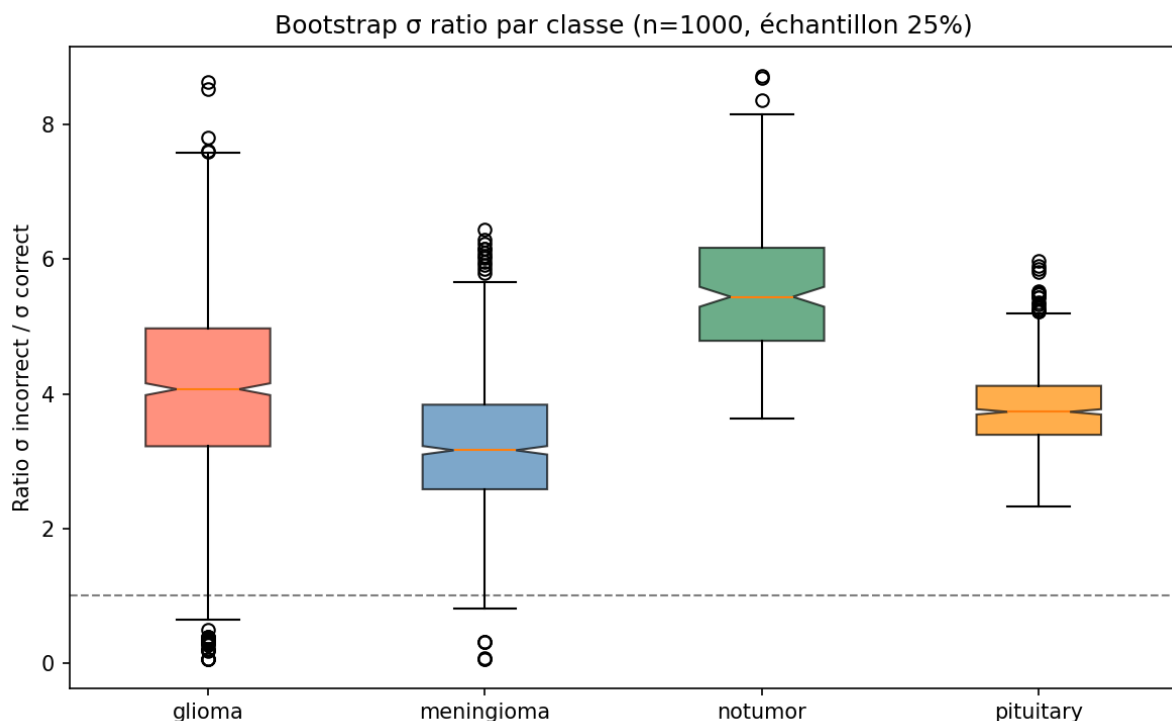


Figure 25 : Bootstrap du ratio $\sigma_{incorrect} / \sigma_{correct}$ par classe (1 000 itérations sur 25 % du jeu de test). La ligne pointillée à 1 représente l'absence de différence entre $\sigma_{incorrect}$ et $\sigma_{correct}$.

VI.G. Calibration bayésienne

Le reliability diagram (diagramme de fiabilité) montre, pour chaque niveau de confiance affiché par le modèle, la précision réelle observée. Un modèle parfaitement calibré aurait tous ses points sur la diagonale.

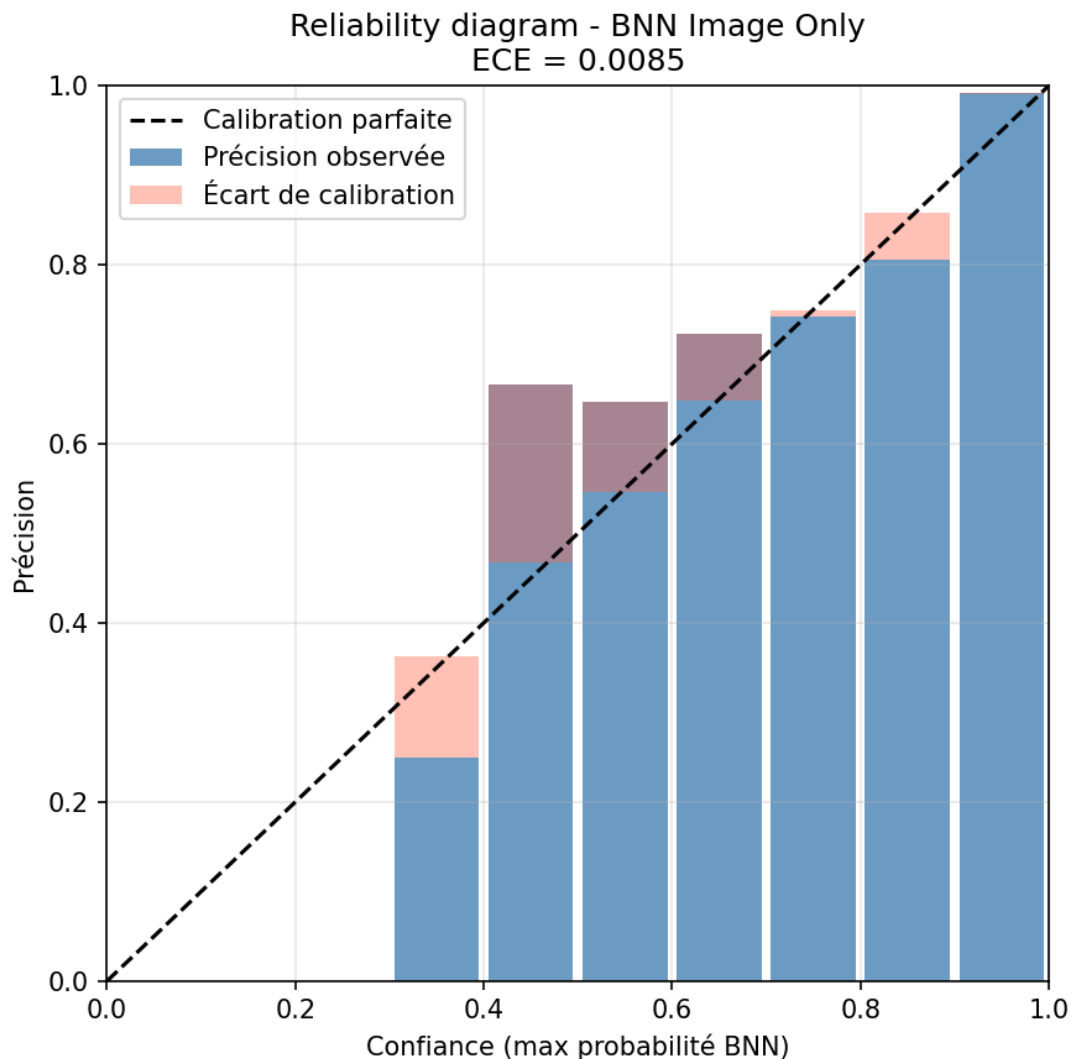


Figure 26 : Reliability diagram du BNN.

Le BNN est mieux calibré que le CNN (ECE 0.0085 contre 0.0160) (figure 26). Les deux restent bien en dessous des ECE entre 0.04 et 0.08 mesurées par Guo et al. (16) sur des CNN standards (cela est probablement dû au fait qu'il n'y a « que » 4 classes).

Le modèle ne produit aucune prédiction avec moins de 30 % de confiance. Les légères oscillations entre calibration et sur-calibration restent dans un intervalle étroit. Au-dessus de 90 %, les prédictions sont bien calibrées. L'ECE de 0.0085 confirme cette bonne calibration globale.

VI.H. Explicabilité visuelle : Grad-CAM

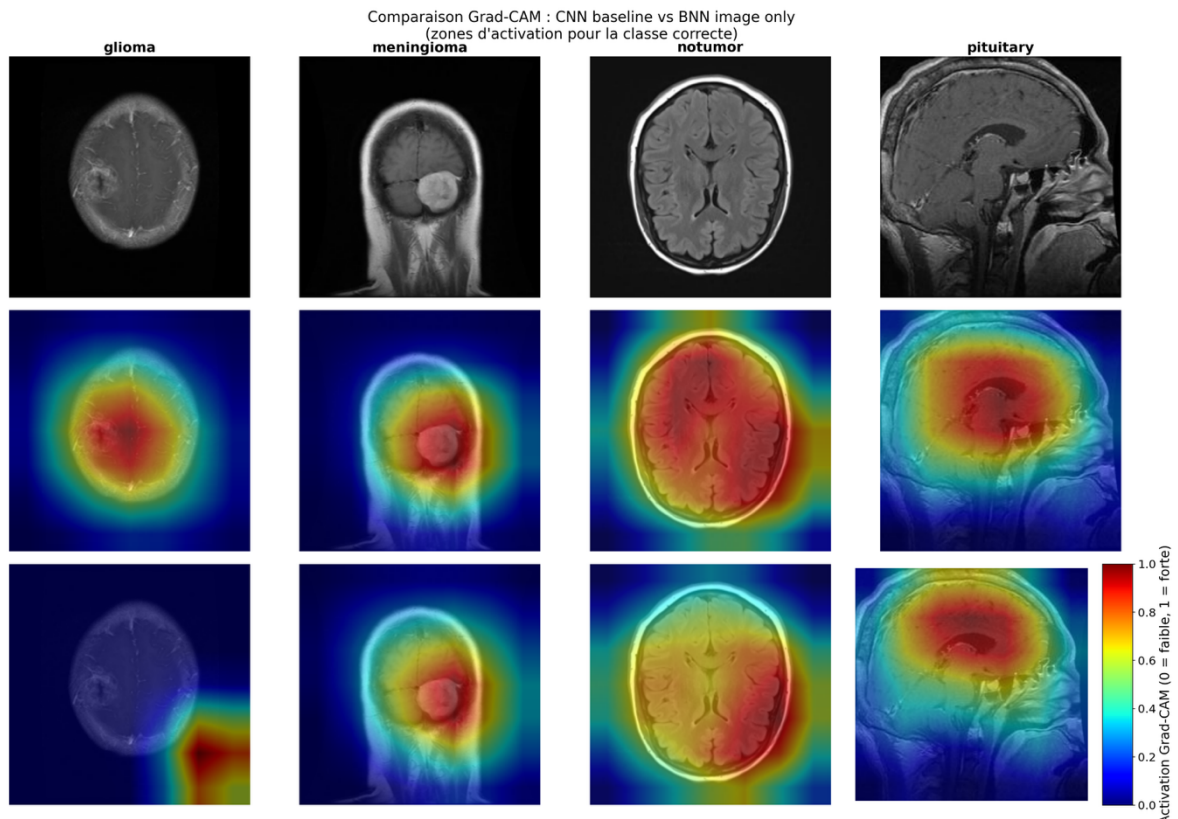


Figure 27 : Comparaison Grad-CAM CNN vs BNN. Première ligne : images originales ; Deuxième ligne : cartes Grad-CAM du CNN ; Troisième ligne : cartes Grad-CAM du BNN. Les zones rouges correspondent aux activations les plus fortes, les zones bleues aux moins fortes.

Pour la plupart des classes les cartes sont similaires. Sur le méningiome et la pituitaire, les deux modèles activent bien la zone tumorale. Sur le gliome, le BNN active une zone décalée par rapport à la tumeur, contrairement au CNN qui se concentre davantage sur celle-ci. Les tumeurs pituitaires se développent dans l'hypophyse (à la base du crâne), et sur la carte de chaleur correspondante l'activation est bien sur cette zone (figure 27).

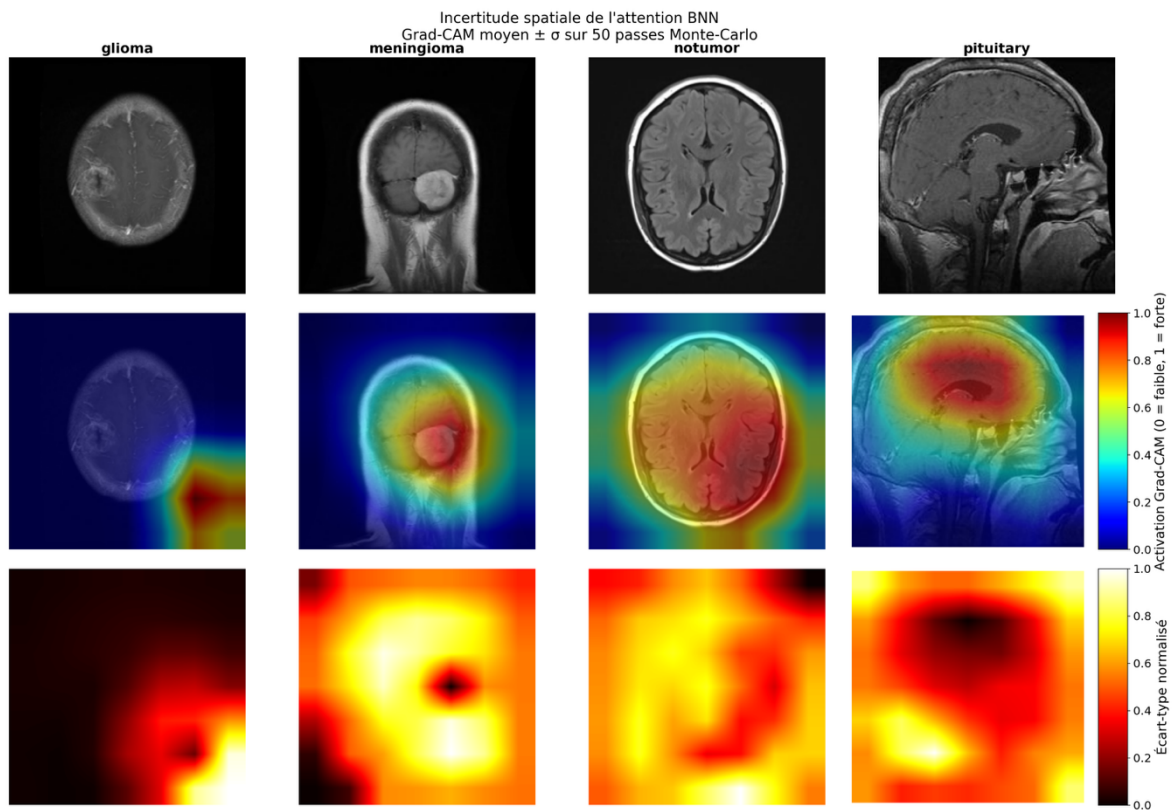


Figure 28 : Carte de l'incertitude spatiale de l'attention du BNN. Première ligne : images originales ; Deuxième ligne : cartes Grad-CAM du BNN ; Troisième ligne : cartes σ . Pour les cartes σ , les zones claires indiquent une forte variabilité de l'attention entre les passes.

Sur la carte d'incertitude, le notumor présente la variabilité la plus étendue sur toute l'image. Pour le méningiome, l'attention est stable sur la tumeur elle-même mais variable autour. Pour la pituitaire, l'incertitude est concentrée en bas de l'image, là où se développe la tumeur. Pour le gliome, la variabilité est localisée en bas de l'image (figure 28).

VI.I. Validation quantitative des Grad-CAM avec masques

Les cartes Grad-CAM sont comparées aux masques de segmentation de Br35H sur 200 images annotées (tableau 3).

Modèle	IoU	Pointing game	Energy ratio
CNN	0.228	0.760	0.170
BNN (carte moyenne)	0.172	0.435	0.135
BNN (carte variance)	0.073	0.150	0.078

Tableau 3 : Validation quantitative des Grad-CAM sur 200 images annotées Br35H.

Le CNN obtient les meilleurs scores sur les trois métriques. La carte d'attention moyenne du BNN est légèrement en dessous, ce qui suggère que la tête bayésienne diffuse un peu l'attention par rapport au CNN. La carte de variance obtient les scores les plus faibles, ce qui est cohérent : elle ne montre pas où est la tumeur mais où l'attention du modèle varie entre les passes.

Le CNN localise mieux la tumeur que le BNN sur ces trois métriques. C'est le compromis du passage au bayésien : on gagne en incertitude et en calibration, mais on perd un peu en précision de localisation.

Quelques exemples qualitatifs illustrent ces résultats :

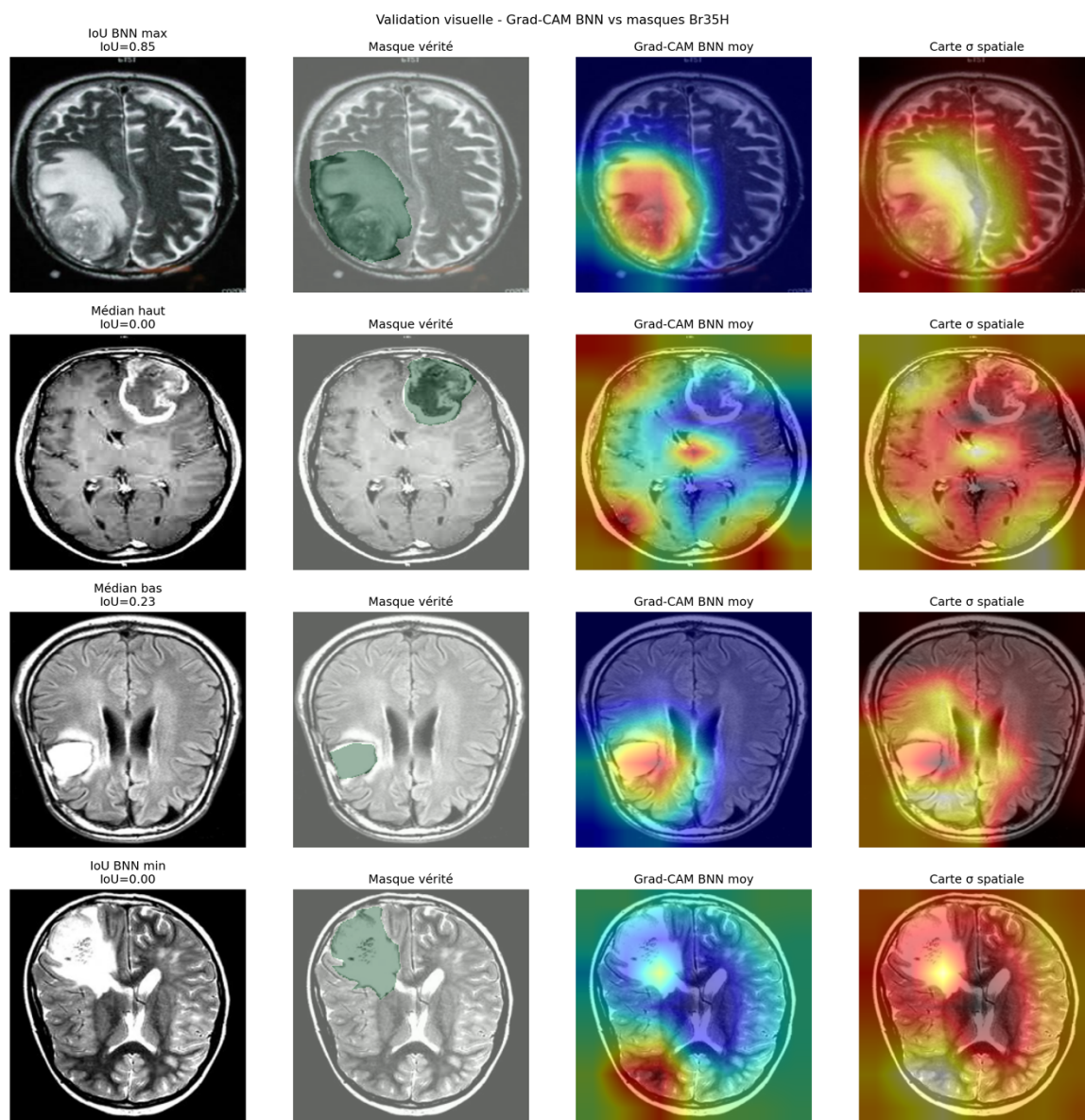


Figure 29 : Exemples qualitatifs de validation Grad-CAM sur Br35H. Pour chaque image : image originale, masque de segmentation, carte Grad-CAM et superposition.

VI.J. BNN Image Only vs multimodal

Rappel : le BNN Image Only a la même architecture que le BNN multimodal mais sans la branche clinique, pour mesurer l'apport réel de la feature d'âge.

Modèle	AUC	Balanced Accuracy	ECE	Temps d'inférence
CNN Baseline	0.998	0.966	0.0160	2.3 ms
BNN Image Only	0.995	0.967	0.0085	102.6 ms
BNN Multimodal	0.995	0.954	0.0276	99.2 ms

Tableau 4 : Récapitulatif des performances et temps d'inférence CNN Baseline / BNN Image Only / BNN Multimodal.

L'AUC est quasi-identique (0.995 dans les deux cas) mais la balanced accuracy est plus faible avec la feature clinique, 0.954 contre 0.967 (tableau 4). L'explication la plus probable est que l'âge est lui-même prédit depuis l'image et apporte une information redondante avec les features visuelles. Les quelques millisecondes d'écart entre l'image-seule et le multimodal changent d'un test à l'autre, la branche clinique ne coûtant presque rien en temps de calcul.

VI.K. Comparaison torchbnn vs MC Dropout

Rappel : torchbnn modélise les poids comme des distributions, MC Dropout utilise un masque aléatoire maintenu actif à l'inférence. J'ai entraîné trois variantes de MC Dropout avec des taux différents (0.2, 0.3, 0.5) car l'incertitude estimée dépend fortement de ce paramètre arbitraire.

On parle d'in-distribution pour désigner les données du Mendeley BT-MRI, c'est-à-dire la même distribution que celle utilisée à l'entraînement (par opposition aux données OOD, voir III.C.2).

Modèle	AUC	Balanced Accuracy	ECE	σ médian in-dist	Ratio σ
CNN Baseline	0.998	0.966	0.0160	-	-
BNN torchbnn	0.995	0.967	0.0085	0.0217	4.83
MC Dropout taux=0.2	0.998	0.974	0.0162	0.0007	8.32
MC Dropout taux=0.3	0.998	0.968	0.0104	0.0010	8.24
MC Dropout taux=0.5	0.999	0.967	0.0105	0.0020	9.41

Tableau 5 : Comparaison torchbnn vs MC Dropout en in-distribution.

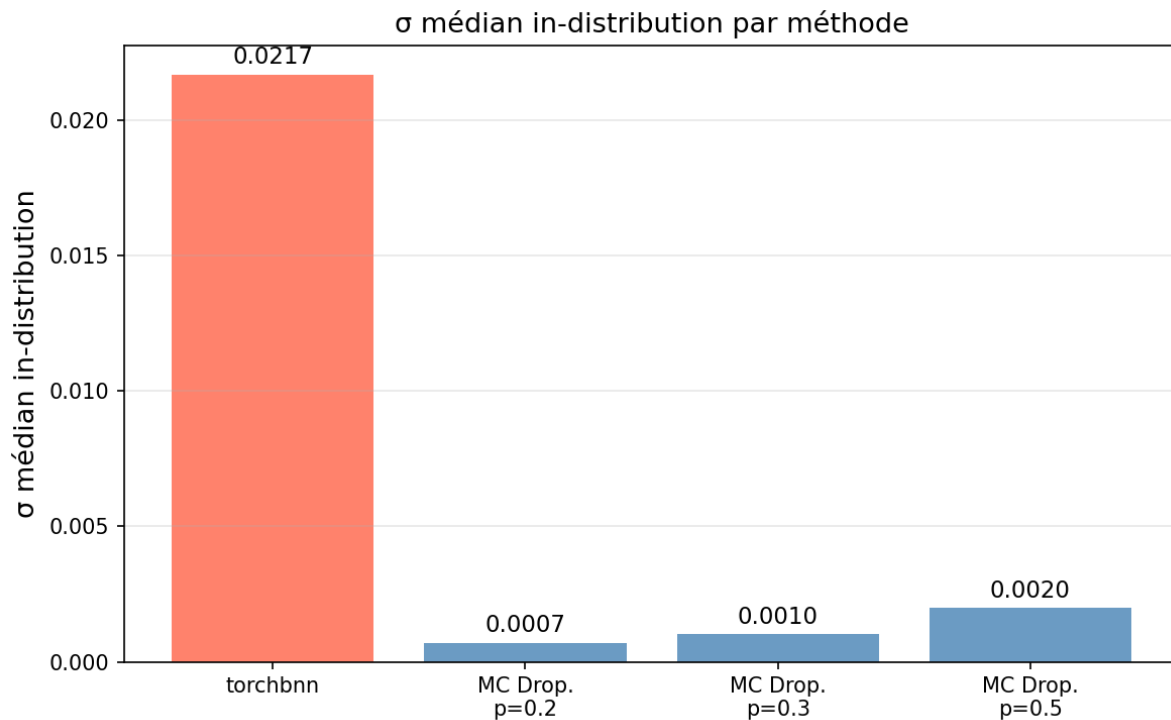


Figure 30 : Comparaison des distributions σ in-distribution pour torchbnn et MC Dropout aux taux 0.2, 0.3 et 0.5.

MC Dropout obtient une AUC légèrement meilleure que torchbnn, pour une balanced accuracy équivalente (0.967 à 0.974 contre 0.967) (tableau 5). torchbnn est un peu mieux calibré sur cette seed (ECE 0.0085 contre 0.0104 à 0.0162). Le σ médian in-distribution de MC Dropout est quasi nul : 0.0007 à 0.0020 selon le taux, contre 0.0217 pour torchbnn (figure 30). MC Dropout est presque déterministe en pratique et ne permet pas de trier les prédictions par incertitude, malgré un ratio $\sigma_{\text{incorrect}} / \sigma_{\text{correct}}$ plus élevé (8.24 à 9.41 contre 4.83).

VI.L. Seuil de triage par incertitude

torchbnn produisant un σ exploitable (tableau 5, VI.K), on peut s'en servir pour trier les prédictions par incertitude croissante et définir un seuil de triage clinique. J'utilise ici l'accuracy (la proportion de prédictions correctes), pas la balanced accuracy des tableaux 4 et 5.

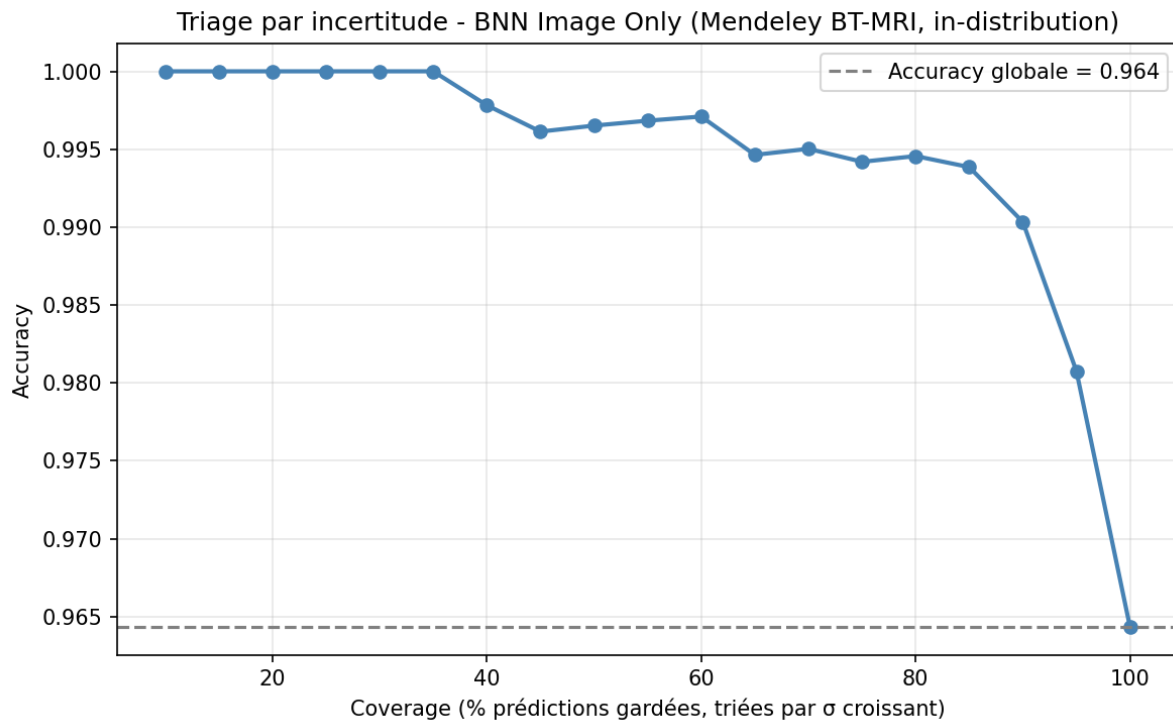


Figure 31 : Courbe accuracy vs coverage sur le jeu de test Mendeley BT-MRI pour torchbnn. En gardant uniquement les prédictions les plus confiantes (faible σ), l'accuracy augmente progressivement. La ligne pointillée représente l'accuracy globale (0.964).

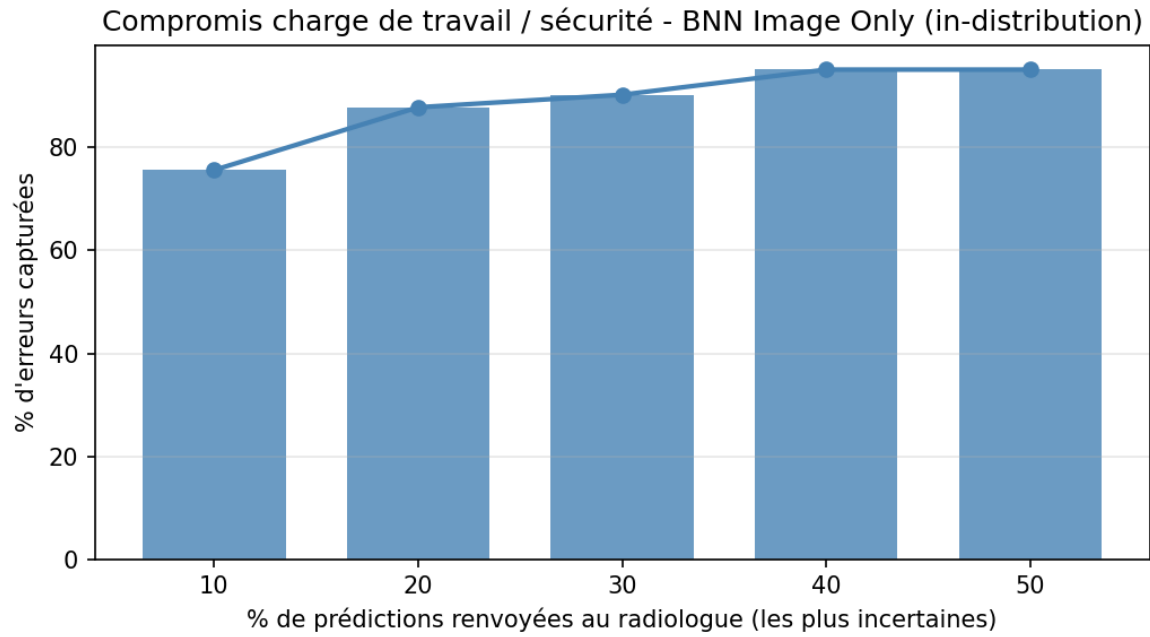


Figure 32 : Compromis charge de travail / sécurité pour torchbnn sur le jeu de test Mendeley BT-MRI. Chaque barre indique le pourcentage d'erreurs capturées en renvoyant les X % de prédictions les plus incertaines à un radiologue.

En ne gardant que les 50 % de prédictions les plus confiantes, l'accuracy passe de 0.964 à 0.997 (figure 31). Renvoyer les 20 % les plus incertaines à un radiologue couvrirait 87.8 % des erreurs du modèle (figure 32). En pratique, un radiologue qui ne vérifie qu'une image sur cinq couvrirait presque neuf erreurs sur dix.

VI.M. Robustesse inter-seed

Tous les résultats précédents sont calculés avec la seed 42. Pour vérifier qu'ils ne dépendent pas de ce choix, j'ai réentraîné de zéro le BNN Image Only (torchbnn) et le MC Dropout (taux 0.3) sur cinq seeds (0, 1, 2, 3 et 42).

Modèle	Seed	ECE	σ médian in-dist
torchbnn	0	0.0068	0.0158
	1	0.0134	0.0277
	2	0.0122	0.0302
	3	0.0240	0.0613
	42	0.0085	0.0217
	Moyenne	0.0130 ± 0.0060	0.0313 ± 0.0158
MC Dropout taux=0.3	0	0.0132	0.0002
	1	0.0141	0.0004
	2	0.0138	0.0002
	3	0.0161	0.0001
	42	0.0104	0.0010
	Moyenne	0.0135 ± 0.0018	0.0004 ± 0.0003

Tableau 6 : ECE et σ médian in-distribution du BNN Image Only (torchbnn) et de MC Dropout sur cinq seeds. La dernière ligne de chaque modèle donne la moyenne $\pm$ l'écart-type sur les cinq seeds.

Sur les cinq seeds, l'AUC reste stable (0.995 pour torchbnn, 0.998 pour MC Dropout). Le σ médian de MC Dropout reste quasi nul (0.0001 à 0.0010) contre 0.0158 à 0.0613 pour torchbnn. torchbnn garde donc une incertitude suffisante pour trier les prédictions sur les cinq seeds, alors que MC Dropout reste presque déterministe. Pour l'ECE, les deux modèles se recouvrent : torchbnn va de 0.0068 à 0.0240 et MC Dropout de 0.0104 à 0.0161 (tableau 6). L'avantage marqué de la seed 42 (VI.K) se réduit sur les autres seeds. torchbnn garde une petite avance en moyenne (0.0130 contre 0.0135) et sur quatre seeds sur cinq, mais l'écart est faible et les plages se recouvrent.

MC Dropout n'apparaît pas dans le tableau suivant : son σ étant presque nul, trier ses prédictions par incertitude n'a pas de sens. Pour torchbnn, le ratio $\sigma_{\text{incorrect}} / \sigma_{\text{correct}}$ et le triage restent stables sur les cinq seeds. L'accuracy varie peu : 0.957 à 0.964 en global, 0.991 à 0.998 sur la moitié la plus confiante. Le ratio et la couverture des erreurs varient davantage. Le ratio va de 2.15 à 4.83 (moyenne 3.69) et les 20 % les plus incertains couvrent de 72.7 % à 87.8 % des erreurs (moyenne 83.2 %) (tableau 7). Les valeurs annoncées dans ce mémoire sont celles de la seed 42, dans le haut de cette page. Le ratio reste néanmoins supérieur à 2 sur toutes les seeds : les prédictions incorrectes restent plus incertaines que les correctes, et le triage couvre toujours la majorité des erreurs.

Seed	Ratio σ	Accuracy globale	Accuracy 50 % confiantes	Erreurs couvertes (20 % les + incertains)
0	4.47	0.958	0.998	85.4 %
1	3.52	0.957	0.995	83.7 %
2	3.49	0.962	0.997	86.4 %
3	2.15	0.962	0.991	72.7 %
42	4.83	0.964	0.997	87.8 %
Moyenne	3.69 ± 0.93	0.961 ± 0.002	0.996 ± 0.002	$83.2 \% \pm 5.4 \%$

Tableau 7 : Ratio $\sigma_{incorrect} / \sigma_{correct}$ et performances de triage du BNN Image Only (torchbnn) sur cinq seeds. La dernière ligne donne la moyenne $\pm$ l'écart-type sur les cinq seeds.

VI.N. Validation externe sur Br35H

Rappel : les trois modèles sont évalués sur Br35H en zero-shot : ils n'ont jamais vu ces images pendant l'entraînement.

Modèle	AUC	Balanced Accuracy
CNN Baseline	0.945	0.809
BNN torchbnn	0.895	0.742
MC Dropout taux=0.3	0.939	0.796

Tableau 8 : Performances OOD sur Br35H (binaire tumeur/pas tumeur).

Les trois modèles sont moins bons que sur le Mendeley BT-MRI, ce qui est attendu sur des données jamais vues. Le CNN conserve la meilleure AUC (0.945) et les balanced accuracy vont de 0.742 à 0.809 (tableau 8).

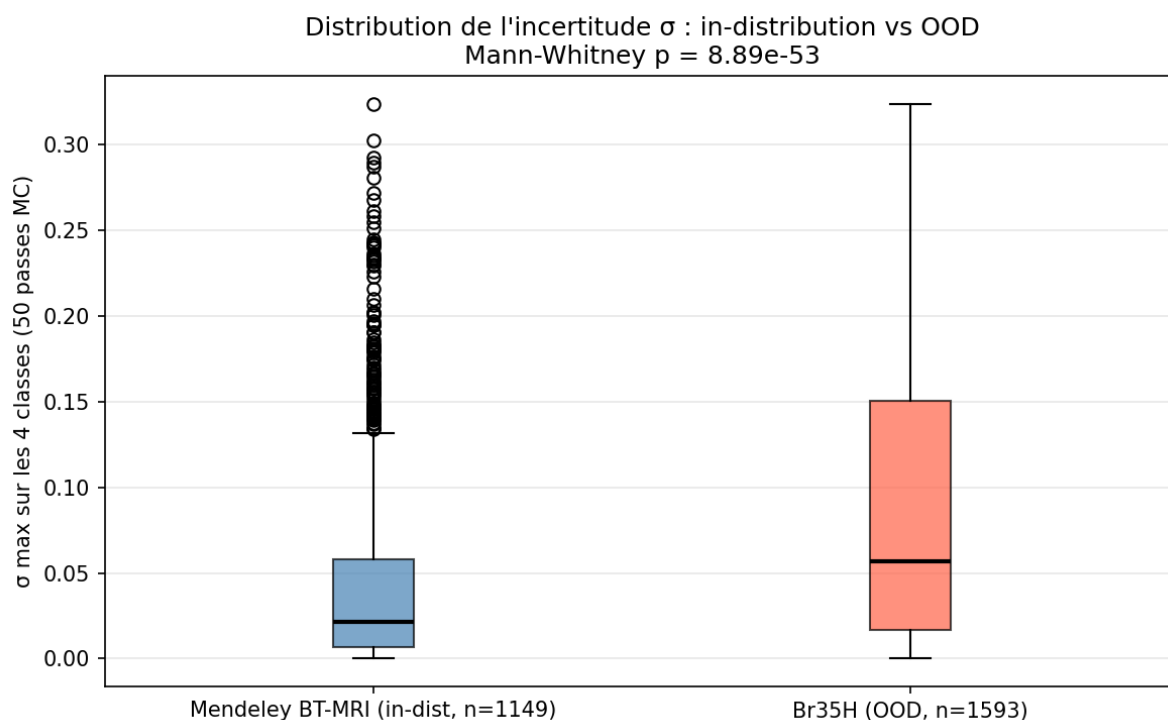


Figure 33 : Distribution de l'incertitude σ sur le Mendeley BT-MRI (in-distribution) et Br35H (OOD) pour le BNN torchbnn.

Le σ médian passe de 0.0217 sur le Mendeley BT-MRI à 0.0570 sur Br35H pour torchbnn, soit un ratio de 2.63, confirmé par un test de Mann-Whitney ($p=8.89e-53$) (figure 33).

Modèle	AUC val	AUC OOD	BalAcc OOD	Ratio σ OOD	p-value
torchbnn	0.995	0.895	0.742	2.63	8.89e-53
MC Dropout taux=0.2	0.998	0.905	0.808	27.27	5.60e-148
MC Dropout taux=0.3	0.998	0.939	0.796	29.90	1.37e-141
MC Dropout taux=0.5	0.998	0.946	0.824	18.90	1.40e-124

Tableau 9 : Comparaison des taux de dropout sur Br35H. AUC val = AUC sur le Mendeley BT-MRI ; AUC OOD = AUC sur Br35H.

MC Dropout affiche des ratios OOD plus élevés que torchbnn (18.90 à 29.90 contre 2.63) (tableau 9), mais son σ in-distribution est quasi nul (0.0007 à 0.0020). L'augmentation du σ sur Br35H part donc d'une base très basse. Pour torchbnn, le σ passe de 0.0217 à 0.0570. Le σ in-distribution de MC Dropout reste inférieur à 0.0020, trop bas pour servir de référence clinique.

VII. Discussion

VII.A. L'incertitude bayésienne est-elle utile ?

Le ratio de 4.83 entre $\sigma_{\text{incorrect}}$ et σ_{correct} est le résultat le plus utile cliniquement. En renvoyant 20 % des prédictions les plus incertaines à un radiologue, on couvrirait 87.8 % des erreurs du modèle. Avec un CNN classique on ne pourrait pas faire cela car il reste confiant même quand il se trompe (16).

Sur le Mendeley BT-MRI, l'incertitude varie aussi selon la classe : le méningiome présente la médiane σ la plus élevée (0.0268), ce qui correspond à sa plus grande difficulté de classification.

Sur Br35H, le σ médian est 2.63 fois plus élevé que sur le Mendeley BT-MRI, une différence confirmée par un test de Mann-Whitney ($p=8.89e-53$). Le modèle est donc plus incertain sur des données qu'il n'a jamais vues, ce qui permet de les identifier avant de rendre une prédiction.

VII.B. torchbnn vs MC Dropout

MC Dropout obtient une AUC légèrement meilleure que torchbnn, une balanced accuracy équivalente, et un ratio $\sigma_{\text{incorrect}} / \sigma_{\text{correct}}$ plus élevé (8.24 à 9.41 contre 4.83). Mais son σ médian in-distribution est quasi nul : 0.0007 à 0.0020 selon le taux, contre 0.0217 pour torchbnn. La différence absolue entre $\sigma_{\text{incorrect}}$ et σ_{correct} est trop petite pour définir un seuil de triage clinique. Sur la seed 42, torchbnn est nettement mieux calibré (ECE 0.0085). L'avantage se réduit sur les autres seeds. torchbnn garde une petite avance en moyenne (0.0130 contre 0.0135) et sur quatre seeds sur cinq, mais elle est faible et les plages se recouvrent.

Sur Br35H, MC Dropout affiche des ratios plus élevés (18.90 à 29.90 contre 2.63), mais son σ in-distribution part d'une base quasi nulle. Pour torchbnn, le σ passe de 0.0217 à 0.0570, soit une hausse de 0.035 en valeur absolue. MC Dropout part d'une base in-distribution quasi nulle (0.001 à 0.002), donc son ratio élevé correspond en fait à un petit écart.

torchbnn produit une incertitude suffisamment élevée pour définir un seuil de triage, sans hyperparamètre arbitraire à régler.

VII.C. Pourquoi les modèles sont bien calibrés ?

Deux raisons : d'abord Kristiadi et al. (28) expliquent qu'une tête bayésienne améliore naturellement la calibration même si le reste du réseau reste déterministe, ce qui explique aussi pourquoi le BNN est mieux calibré que le CNN (ECE 0.0085 contre 0.0160). Ensuite,

avec seulement 4 classes assez différentes et des AUC très élevées (0.998 pour le CNN, 0.995 pour le BNN), le modèle sort naturellement des probabilités proches de 0 ou de 1 sur les bonnes classes, ce qui facilite la calibration.

VII.D. Apport de la feature clinique

Le BNN multimodal fait moins bien que le BNN Image Only : 0.954 de balanced accuracy contre 0.967, pour une AUC identique (0.995) et une calibration moins bonne (ECE 0.0276 contre 0.0085). Comme l'âge est lui-même prédit depuis l'IRM, il apporte la même information que les features visuelles, donc son ajout n'améliore pas le modèle. D'autres variables cliniques indépendantes de l'image, comme l'âge réel ou des données biologiques, pourraient apporter un gain.

VII.E. Comparaison avec les approches existantes

Plusieurs travaux ont utilisé des datasets similaires pour la classification de tumeurs cérébrales. Sur des datasets à 3 ou 4 classes proches du dataset Mendeley BT-MRI, des approches de transfer learning atteignent déjà 0.99, l'objectif de ce mémoire n'est pas de dépasser ces performances mais d'améliorer l'explicabilité et la fiabilité des prédictions.

Dans les travaux que j'ai consultés, aucun modèle n'intègre ces trois dimensions simultanément : performance, incertitude calibrée, et explicabilité visuelle. La plupart se concentrent sur l'AUC sans s'intéresser à la fiabilité des prédictions, et ce type de ratio $\sigma_{\text{incorrect}} / \sigma_{\text{correct}}$ sur des datasets équivalents n'y apparaît pas. Produire une carte d'incertitude spatiale de l'attention (écart-type des cartes Grad-CAM sur 50 passes MC) apparaît également peu dans la littérature : Roy et al. (19) et Natekar et al. (21) utilisent l'incertitude MC pour la segmentation, mais pas pour cartographier l'incertitude spatiale de l'attention. Les scores de localisation de cette carte restent faibles.

La comparaison directe torchbnn contre MC Dropout sur les mêmes données avec validation externe est aussi peu présente dans la littérature sur la classification IRM. La plupart des travaux utilisent MC Dropout sans le comparer à l'inférence variationnelle ni tester la détection du changement de distribution.

L'utilisation d'un âge prédit plutôt que réel est une contrainte du dataset, pas un choix de design. La plupart des architectures multimodales en imagerie médicale utilisent des données patient directes (âge, sexe, biomarqueurs).

VII.F. Limites

VII.F.1. Limites méthodologiques

Coupe 2D vs volume 3D

Le pipeline fonctionne sur des coupes 2D isolées, alors qu'un radiologue analyse toujours la série complète. Une lésion peut être plus ou moins visible selon la coupe choisie, et les informations des coupes voisines ne sont pas utilisées. Cette limite explique en partie le MAE de 6.1 ans de BrainAgeNet, l'âge étant mieux estimé sur des volumes 3D complets (18).

Validation Grad-CAM partielle

La validation quantitative sur Br35H (IoU, pointing game, energy ratio) apporte des chiffres, mais les masques ne distinguent pas les types de tumeurs. Une validation par des radiologues sur Mendeley BT-MRI reste nécessaire pour conclure.

Grad-CAM et causalité

Les cartes Grad-CAM montrent des corrélations entre des zones et la décision, mais pas nécessairement une causalité. Le modèle pourrait activer des zones corrélées à la classe sans que ce soit la vraie cause de la prédiction.

Apport du multimodal limité

La feature d'âge n'améliore pas le modèle. L'architecture multimodale reste pertinente, mais il faudrait des variables cliniques indépendantes de l'image, qu'il n'y a pas dans ce dataset.

VII.F.2. Limites techniques

Robustesse inter-seed

J'ai réentraîné le BNN Image Only et le MC Dropout sur cinq seeds, mais je n'ai mesuré la détection OOD et les ratios par classe que sur la seed 42. Son ratio $\sigma_{\text{incorrect}} / \sigma_{\text{correct}}$ est de 4.83 pour une moyenne de 3.69 sur les cinq seeds, et son triage couvre 87.8 % des erreurs pour une moyenne de 83.2 %. Sur les autres seeds le ratio reste au-dessus de 2 et le triage couvre toujours la majorité des erreurs, mais les chiffres que je mets en avant viennent d'une seed favorable.

Temps d'inférence

Le temps d'inférence avec 50 passes MC est de 102.6 ms par image, contre 2.3 ms pour le CNN baseline, soit environ 45 fois plus lent. En pratique un radiologue ne consulte pas les images en temps réel mais après acquisition, et avec environ 7 millions d'IRM réalisées chaque année en France (29), quelques centaines de millisecondes par image sont largement acceptables.

VII.F.3. Limites de généralisation

Taille du dataset

Le dataset propre ne compte que 4 606 images d'entraînement, dont seulement 592 pour la classe notumor. C'est peu pour du deep learning, et je n'ai pas vérifié si ces résultats tiendraient avec plus de données, ou sur un autre dataset à 4 classes en in-distribution.

Biais du dataset

La provenance du dataset Mendeley BT-MRI n'est pas documentée. La performance du modèle sur des données acquises avec d'autres équipements d'IRM ou dans des contextes démographiques et cliniques différents n'est pas garantie et nécessiterait une validation préalable à tout déploiement clinique.

Validation externe limitée

La validation externe est limitée à un seul dataset binaire. Br35H ne distingue pas les types de tumeurs, donc on ne peut pas mesurer si le modèle se trompe de type sur des données nouvelles.

Déséquilibre du jeu OOD après nettoyage

Le nettoyage déséquilibre le jeu OOD. Au départ il est équilibré 50/50, mais après nettoyage il reste environ 82 % d'images avec tumeur et 18 % sans tumeur. La balanced accuracy limite l'effet de ce déséquilibre, mais le ratio σ OOD est calculé sur ce jeu déséquilibré.

VII.F.4. Limites cliniques et éthiques

Biais de validation par des radiologues

Une validation par des radiologues introduirait un biais si ceux-ci connaissent les prédictions du modèle avant de donner leur avis. Un protocole en double aveugle serait nécessaire : le radiologue ne verrait pas la prédiction du modèle avant d'avoir donné son propre diagnostic. C'est faisable mais coûteux en temps et en ressources.

Consentement

Un déploiement clinique poserait aussi des questions de consentement : certains patients sont décédés, et les familles n'accepteraient pas forcément que leurs données servent à entraîner une IA. D'autant plus que l'IA reste parfois mal perçue par le grand public.

VII.G. Déploiement clinique

Dans un contexte clinique, l'incertitude peut servir de triage. Les images sur lesquelles le modèle est incertain (σ élevé) seraient vérifiées en priorité par un radiologue. Cela est conforme à l'AI Act sur la supervision humaine : le système aide mais ne décide pas seul.

Sur le Mendeley BT-MRI, renvoyer 20 % des prédictions les plus incertaines à un radiologue couvrirait 87.8 % des erreurs du modèle. L'ECE de 0.0085 montre que si le modèle affiche 80 % de confiance, il est correct environ 80 % du temps. La carte d'incertitude spatiale apporte une information supplémentaire : le radiologue peut voir non seulement que le modèle est incertain, mais aussi sur quelle zone de l'IRM.

Un σ très élevé signale une image que le modèle n'a probablement jamais vue. Ces images peuvent être renvoyées au radiologue sans que le modèle ne rende de prédiction. Les cas ainsi annotés par le radiologue deviennent de nouvelles données d'entraînement. En réentraînant le modèle périodiquement sur ces données, il pourrait s'améliorer sur les cas qu'il traite mal et s'adapter à de nouveaux équipements ou populations.

VII.H. Perspectives et travaux futurs

La validation externe pourrait être étendue à plusieurs datasets de différentes origines géographiques et constructeurs IRM, pour mieux mesurer la généralisation du modèle.

Étendre la détection OOD et les ratios par classe sur plusieurs seeds donnerait des intervalles plutôt que les valeurs d'une seule initialisation, et confirmerait que la seed présentée n'est pas un cas particulier.

On pourrait aussi étendre l'approche à plusieurs coupes 2D par patient, avant de passer à des volumes 3D complets, afin d'exploiter plus d'information par patient.

Pour la validation Grad-CAM, des annotations par des radiologues sur Mendeley BT-MRI permettraient une évaluation quantitative, plus représentative que les masques binaires de Br35H.

On pourrait aussi utiliser un dataset qui inclut l'âge réel des patients, ou d'autres variables cliniques comme des données biologiques, pour voir si la contribution de la feature clinique est plus importante quand elle est moins approximative.

Ajouter plusieurs variables cliniques simultanément permettrait d'analyser leur contribution respective et de réduire certains biais liés au dataset.

Sur le plan architectural, l'inférence variationnelle pourrait être étendue à tout le réseau et pas seulement à la tête. Cette approche pourrait aussi s'appliquer à d'autres types d'imagerie médicale.

VIII. Conclusion

Ce mémoire cherche à répondre à la question suivante : **comment améliorer l'explicabilité et la fiabilité d'un modèle de classification d'IRM cérébrales, et dans quelle mesure l'incertitude bayésienne apporte-t-elle un avantage par rapport aux approches classiques ?**

Cette question a conduit à développer et évaluer une architecture BNN combinant les features visuelles d'un ResNet18 avec une tête bayésienne, à tester une variante multimodale avec une estimation de l'âge (qui n'a finalement pas amélioré les résultats), puis à comparer deux approches bayésiennes (torchbnn et MC Dropout) avec une validation externe sur Br35H. J'ai aussi découvert que 61.6 % des images de test étaient présentes dans le train. J'ai donc reconstruit un split propre et ré-entraîné tous les modèles sur ces données corrigées.

Pour y répondre :

Le modèle produit-il une incertitude utile ?

Oui, les prédictions incorrectes sont 4.83 fois plus incertaines que les correctes. Par classe, le ratio est de 4.39 pour le gliome et 3.67 pour le méningiome, les deux classes avec assez d'erreurs pour que la moyenne soit représentative. En pratique, renvoyer les 20 % de prédictions les plus incertaines à un radiologue couvrirait 87.8 % des erreurs du modèle. Sur les cinq seeds, le ratio reste supérieur à 2 et le triage couvre toujours la majorité des erreurs, mais la seed 42 donne les chiffres les plus élevés, 4.83 et 87.8 %.

L'inférence variationnelle est-elle préférable à MC Dropout ?

Sur le Mendeley BT-MRI, MC Dropout fait légèrement mieux sur les performances brutes. Mais son incertitude en in-distribution est presque nulle (σ médian 0.0010 contre 0.0217 pour torchbnn), ce qui ne permet pas de définir un seuil de triage clinique. Ce σ presque nul se retrouve sur les cinq seeds. torchbnn produit une incertitude assez élevée pour le triage. Pour un usage clinique réel, torchbnn est le meilleur choix.

La confiance est-elle fiable ?

Oui, l'ECE de 0.0085 reste bien en dessous des valeurs mesurées sur des CNN standards (0.04 à 0.08), et le BNN est mieux calibré que le CNN baseline (0.0160).

Le modèle généralise-t-il à d'autres données ?

En partie. Sur Br35H, les performances baissent (AUC 0.895 contre 0.995) mais l'incertitude augmente nettement (σ médian 2.63 fois plus élevé, $p=8.89e-53$), ce qui permet de détecter les images hors distribution.

L'attention est-elle cohérente visuellement ?

En partie. Les cartes Grad-CAM du CNN localisent bien la tumeur, mais le BNN diffuse un peu l'attention et obtient des scores de localisation plus faibles. C'est le compromis du passage au bayésien.

Peut-on aller plus loin que la carte d'attention moyenne ?

Oui, l'écart-type pixel par pixel des 50 cartes Grad-CAM produit une carte de variance spatiale, qui montre où l'attention du modèle varie le plus entre les passes, ce que la carte moyenne ne montre pas, mais ses scores de localisation restent faibles.

Par rapport au CNN baseline, le BNN apporte une mesure d'incertitude par prédiction, une meilleure calibration, et une carte d'attention dont on peut mesurer la variabilité entre les passes. Ces dimensions combinées permettent de rendre un modèle de classification IRM à la fois performant, transparent et conscient de ses limites, ce qui est exactement ce qu'on attend d'un système d'aide au diagnostic clinique.

Cette approche répond également aux exigences de transparence imposées par l'AI Act pour les systèmes à haut risque, et montre qu'on peut aller plus loin que la simple performance en classification IRM.

IX. Bibliographie

1. European Parliament. Artificial Intelligence Act (Regulation (EU) 2024/1689) [Internet]. Official Journal of the European Union; 2024. Disponible sur: <https://eur-lex.europa.eu/legal-content/FR/TXT/?uri=CELEX:32024R1689>
2. Bray F, Laversanne M, Sung H, Ferlay J, Siegel RL, Soerjomataram I, et al. Global cancer statistics 2022: GLOBOCAN estimates of incidence and mortality worldwide for 36 cancers in 185 countries. *CA Cancer J Clin.* mai 2024;74(3):229-63. doi:10.3322/caac.21834
3. Leung D, Han X, Mikkelsen T, Nabors LB. Role of MRI in primary brain tumor evaluation. *J Natl Compr Cancer Netw JNCCN.* nov 2014;12(11):1561-8. doi:10.6004/jnccn.2014.0156 PubMed PMID: 25361803.
4. Xie Y, Zaccagna F, Rundo L, Testa C, Agati R, Lodi R, et al. Convolutional Neural Network Techniques for Brain Tumor Classification (from 2015 to 2022): Review, Challenges, and Future Perspectives. *Diagnostics.* 31 juill 2022;12(8):1850. doi:10.3390/diagnostics12081850
5. Hegi ME, Diserens AC, Gorlia T, Hamou MF, De Tribolet N, Weller M, et al. *MGMT* Gene Silencing and Benefit from Temozolomide in Glioblastoma. *N Engl J Med.* 10 mars 2005;352(10):997-1003. doi:10.1056/NEJMoa043331
6. Calabrese E, Villanueva-Meyer JE, Rudie JD, Rauschecker AM, Baid U, Bakas S, et al. The University of California San Francisco Preoperative Diffuse Glioma MRI Dataset. *Radiol Artif Intell.* 1 nov 2022;4(6):e220058. doi:10.1148/ryai.220058
7. Gal Y, Ghahramani Z. Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning. In: *Proceedings of The 33rd International Conference on Machine Learning* [Internet]. PMLR; 2016 [cité 17 avr 2026]. p. 1050-9. Disponible sur: <https://proceedings.mlr.press/v48/gal16.html>
8. Kendall A, Gal Y. What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision? In: *Advances in Neural Information Processing Systems* [Internet]. Curran Associates, Inc.; 2017 [cité 17 avr 2026]. Disponible sur: <https://proceedings.neurips.cc/paper/2017/hash/2650d6089a6d640c5e85b2b88265dc2b-Abstract.html>
9. Selvaraju RR, Cogswell M, Das A, Vedantam R, Parikh D, Batra D. Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. In: *2017 IEEE International Conference on Computer Vision (ICCV)* [Internet]. Venice: IEEE; 2017 [cité 17 avr 2026]. p. 618-26. Disponible sur: <http://ieeexplore.ieee.org/document/8237336/> doi:10.1109/ICCV.2017.74
10. HIRA MIK, HOSSAIN MS, BITHEE MMA, Sara US, HASAN MM, Towsif AA, et al. Brain Tumor MRI Dataset (Glioma, Meningioma, Pituitary, No Tumor) [Internet]. Mendeley Data; 2025 [cité 17 avr 2026]. Disponible sur: <https://data.mendeley.com/datasets/zwr4ntf94j/4> doi:10.17632/ZWR4NTF94J.4
11. Zauner C. Implementation and Benchmarking of Perceptual Image Hash Functions.

12. Br35H :: Brain Tumor Detection 2020 [Internet]. [cité 15 mai 2026]. Disponible sur: <https://www.kaggle.com/datasets/ahmedhamada0/brain-tumor-detection>
13. Krizhevsky A, Sutskever I, Hinton GE. ImageNet Classification with Deep Convolutional Neural Networks. In: Advances in Neural Information Processing Systems [Internet]. Curran Associates, Inc.; 2012 [cité 17 avr 2026]. Disponible sur: <https://proceedings.neurips.cc/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html>
14. Tajbakhsh N, Shin JY, Gurudu SR, Hurst RT, Kendall CB, Gotway MB, et al. Convolutional Neural Networks for Medical Image Analysis: Full Training or Fine Tuning? IEEE Trans Med Imaging. mai 2016;35(5):1299-312. doi:10.1109/TMI.2016.2535302
15. He K, Zhang X, Ren S, Sun J. Deep Residual Learning for Image Recognition. In: 2016 [cité 17 avr 2026]. p. 770-8. Disponible sur: https://openaccess.thecvf.com/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html
16. Guo C, Pleiss G, Sun Y, Weinberger KQ. On Calibration of Modern Neural Networks. In: Proceedings of the 34th International Conference on Machine Learning [Internet]. PMLR; 2017 [cité 17 avr 2026]. p. 1321-30. Disponible sur: <https://proceedings.mlr.press/v70/guo17a.html>
17. Blundell C, Cornebise J, Kavukcuoglu K, Wierstra D. Weight Uncertainty in Neural Networks.
18. Cole JH, Poudel RPK, Tsagkrasoulis D, Caan MWA, Steves C, Spector TD, et al. Predicting brain age with deep learning from raw imaging data results in a reliable and heritable biomarker. NeuroImage. déc 2017;163:115-24. doi:10.1016/j.neuroimage.2017.07.059
19. Roy AG, Conjeti S, Navab N, Wachinger C. Bayesian QuickNAT: Model uncertainty in deep whole-brain segmentation for structure-wise quality control. NeuroImage. 15 juill 2019;195:11-22. doi:10.1016/j.neuroimage.2019.03.042
20. Nair T, Precup D, Arnold DL, Arbel T. Exploring uncertainty measures in deep networks for Multiple sclerosis lesion detection and segmentation. Med Image Anal. janv 2020;59:101557. doi:10.1016/j.media.2019.101557
21. Natekar P, Kori A, Krishnamurthi G. Demystifying Brain Tumor Segmentation Networks: Interpretability and Uncertainty Analysis. Front Comput Neurosci. 7 févr 2020;14:6. doi:10.3389/fncom.2020.00006
22. Hochreiter S. The Vanishing Gradient Problem During Learning Recurrent Neural Nets and Problem Solutions. Int J Uncertain Fuzziness Knowl-Based Syst. avr 1998;06(02):107-16. doi:10.1142/S0218488598000094
23. Adaloglou N. AI Summer [Internet]. 2020 [cité 10 juin 2026]. Intuitive Explanation of Skip Connections in Deep Learning. Disponible sur: <https://theaisummer.com/skip-connections/>
24. Li H, Xu Z, Taylor G, Studer C, Goldstein T. Visualizing the Loss Landscape of Neural Nets [Internet]. arXiv; 2018 [cité 10 juin 2026]. Disponible sur: <http://arxiv.org/abs/1712.09913> doi:10.48550/arXiv.1712.09913

25. Kingma DP, Ba J. Adam: A Method for Stochastic Optimization [Internet]. arXiv; 2017 [cité 16 mai 2026]. Disponible sur: <http://arxiv.org/abs/1412.6980> doi:10.48550/arXiv.1412.6980
26. Tan M, Le Q. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. In: Proceedings of the 36th International Conference on Machine Learning [Internet]. PMLR; 2019 [cité 17 avr 2026]. p. 6105-14. Disponible sur: <https://proceedings.mlr.press/v97/tan19a.html>
27. Loshchilov I, Hutter F. Decoupled Weight Decay Regularization [Internet]. arXiv; 2019 [cité 16 mai 2026]. Disponible sur: <http://arxiv.org/abs/1711.05101> doi:10.48550/arXiv.1711.05101
28. Kristiadi A, Hein M, Hennig P. Being Bayesian, Even Just a Bit, Fixes Overconfidence in ReLU Networks. In: Proceedings of the 37th International Conference on Machine Learning [Internet]. PMLR; 2020 [cité 17 avr 2026]. p. 5436-46. Disponible sur: <https://proceedings.mlr.press/v119/kristiadi20a.html>
29. OCDE [Internet]. [cité 17 mai 2026]. Examens d'imagerie par résonance magnétique (IRM). Disponible sur: <https://www.oecd.org/fr/data/indicators/magnetic-resonance-imaging-mri-exams.html>

X. Annexes

L'ensemble des notebooks est disponible sur le dépôt GitHub : <https://github.com/teodulga/memoire-m2-bnn-irm>

Annexe 0 : Notebook 00 - Déduplication et split propre

Déduplication du dataset Mendeley BT-MRI par hachage perceptuel et construction du split propre.

Fichier : 00_clean_split.ipynb

Annexe 1 : Notebook 01 - CNN Baseline (ResNet18)

Entraînement du CNN baseline sur le dataset Mendeley BT-MRI.

Fichier : 01_cnn_baseline.ipynb

Annexe 2 : Notebook 02 - Prédicteur d'âge (BrainAgeNet)

Entraînement du prédicteur d'âge sur le dataset IXI.

Fichier : 02_brain_age_prediction.ipynb

Annexe 3 : Notebook 03 - BNN

Entraînement du BNN Image Only et du BNN multimodal pour comparaison.

Fichier : 03_bnn_multimodal.ipynb

Annexe 4 : Notebook 04 - Analyse de l'incertitude et calibration

Calcul de l'incertitude épistémique par classe, reliability diagram et ECE.

Fichier : 04_clinical_analysis.ipynb

Annexe 5 : Notebook 05 - Grad-CAM et explicabilité visuelle

Génération des cartes Grad-CAM et des cartes d'incertitude spatiale.

Fichier : 05_gradcam.ipynb

Annexe 6 : Notebook 06 - Variante MC Dropout

Entraînement de la variante MC Dropout et comparaison des taux 0.2, 0.3 et 0.5.

Fichier : 06_bnn_mc_dropout.ipynb

Annexe 7 : Notebook 07 - Validation externe Br35H

Évaluation OOD sur Br35H, détection du changement de distribution et validation quantitative des Grad-CAM avec masques de segmentation.

Fichier : 07_br35h.ipynb

Annexe 8 : Notebook 08 - Robustesse inter-seed

Réentraînement du BNN Image Only et de MC Dropout sur cinq seeds pour la sous-section VI.M.

Fichier : 08_seed_robustness.ipynb

Annexe 9 : Notebook 09 - Analyses complémentaires

Bootstrap sur les ratios σ , comparaison ResNet et learning rate, scatter plot BrainAgeNet avec KDE.

Fichier : 09_extra.ipynb